ФСП федерация спортивного программирования России

ФСП Job Board

сопроводительная документация

Цифровая платформа подбора IT-специалистов с верифицированным профилем достижений участника ФСП. Обратная механика: категоризация по адаптивному тесту, выход на контакт по инициативе работодателя.

Команда Full Stack Couple · весь стек разворачивается локально в Docker Compose, без облачных сервисов.

1. Обзор решения и соответствие ТЗ
2. Архитектура решения
3. Механика тестирования
4. Механика подбора
5. Валидация
6. Бенчмарк подбора: методика
7. Бенчмарк подбора: результаты
8. Витрина: ФСП в выдаче
9. Живые вакансии (bench3)
10. Интеграция с ФСП
11. API
12. Компоненты
13. Развёртывание
14. Концепции развития
15. Живой пилот

Версия от 21.10.2026 · документация собрана из репозитория (docs/*.md)

Обзор решения

ФСП Job Board — веб-платформа подбора IT-специалистов с обратной механикой. Кандидат один раз проходит опрос и адаптивный тест на заявленный грейд и получает категорию «специализация × грейд». Работодатель описывает потребность текстом, получает ранжированную подборку с объяснением и сам приглашает конкретного человека, указывая вилку зарплаты. Контакты кандидата открываются только после того, как он принял приглашение или откликнулся сам. Достижения ФСП подтягиваются по ФСП ID и поднимают кандидата в выдаче; если истории ФСП нет, кандидат не получает штрафа.

Всё, что описано в документации, входит в поставку Docker Compose и разворачивается локально одной командой `make up`. Облачные сервисы, ключи и GPU не нужны: серверное ML работает на CPU внутри контейнера `backend`, модели цифрового проктора — в браузере кандидата.

Ключевые факты

Банк заданий	303 параметризованных семейства (шаблона), 31 навык, 7 специализаций, уровни сложности 1–5; правильный ответ вычисляется исполнением Python/SQL; задания на JavaScript, Java, Go, C# и PHP сверены в CI с настоящими компиляторами и интерпретаторами
Тест	IRT 3PL + классификационный CAT: стоп при P ≥ 0.85, 10–25 заданий, в среднем 16 за попытку и 29 за онбординг; для Middle+ — практический кейс из отрасли кандидата
Справочник категорий	7 специализаций × 5 грейдов (Intern, Junior, Middle, Senior, Lead)
Подбор	Формула v7: специфичность требований, «ворота» основного стека, формата и вилки; ФСП — 14% скора и засчитывается как опыт; объяснение у каждого кандидата
Качество подбора	Слепая разметка живых текстов резюме: nDCG@10 0.82 против 0.58 у BM25 и 0.65 у лучшего нейросетевого поиска; в топ-10 в среднем 6.0 подходящих кандидатов против 3.7 и 4.4 (раздел «Бенчмарк подбора: результаты»)
Устойчивость к утечкам	При утечке 50% банка ложные подтверждения грейда — 23% против 60% у фиксированного теста
Платформа	FastAPI (модульный монолит, API без состояния), PostgreSQL 18, Keycloak 26.8, React 19; 72 метода API в OpenAPI; 19 таблиц
Ресурсы	2 vCPU, 4 ГБ RAM, Docker 24+; весь стек ≈ 2 ГБ RAM; запуск — около 5 минут
Качество кода	1030 автотестов ML-ядра и 48 тестов API; линтер ruff; воспроизводимая валидация `make validate`

Соответствие требованиям ТЗ к документации (§3.4)

Требование ТЗ	Раздел
Функциональная и компонентная архитектура	«Архитектура решения», §1–2
Механика тестирования и обоснование устойчивости к распространению заданий	«Механика тестирования», §2–9 (устойчивость — §7)
Механика подбора и логика категоризации	«Механика подбора», §2–8
Собственная процедура валидации и её результаты	«Валидация» (синтетическая популяция со скрытой истиной), «Бенчмарк подбора» (слепая разметка), «Живые вакансии (bench3)»
Схема интеграции с реестром ФСП	«Интеграция с ФСП»
Описание API	«API» (сгенерировано из OpenAPI; Swagger UI — `/api/docs`)
Библиотеки и компоненты с версиями	«Компоненты» (сгенерировано из `uv.lock` и `package-lock.json`)
Пошаговая инструкция по сборке, развёртыванию и локальному запуску	«Развёртывание»

Соответствие функциональным требованиям (§2.1–2.2)

Кабинет кандидата

Требование	Реализация
Профиль вручную или из PDF с автораспознаванием; стандартизированный PDF-профиль	Форма профиля; загрузка PDF → pdfplumber + словарь технологий (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы); генерация PDF-профиля (fpdf2)
Опрос → выбор грейда → тест → категория	Обязательный опрос (отрасль, специализация, направления, стек, условия), адаптивный тест, категория видна работодателю сразу после подтверждения
Регулярные задания	Задания от работодателей с затуханием сигнала (полураспад 60 дней) — компонент силы профиля
Категория и грейд, история, тест выше/ниже с ограничением частоты	История попыток; грейд не понижается; смена грейда — не чаще раза в 90 дней; тест на ступень ниже — сразу после неудачи
Приглашения с вакансией и зарплатой, принять/отклонить	Входящие с вилкой «от–до», компанией и способом связи; отказ с причиной
Вакансии и отклик	Лента вакансий, отсортированная по соответствию, с объяснением; список откликов со статусами
Настройки: приватность, согласия, ФСП ID	Скрытие от отраслей и компаний, скрытие достижений, согласия 152-ФЗ с версиями, привязка ФСП ID
Кандидат видит только свои записи	Чужие объекты → 404; проверено сквозными тестами API

Кабинет работодателя

Требование	Реализация
Профиль компании	Описание, направление, контакты, URL webhook для ATS
Описание потребности → подборка	Разбор текста: стек с важностью, специализация, грейд, опыт, вилка — с возможностью поправить
Подборка с обоснованием	Рекомендованные категории + ранжированные кандидаты, разложение балла и причины
Поиск по банку кандидатов	Фильтры: специализация, грейд, категория, стек, подтверждённые достижения ФСП
Приглашение без привязки к вакансии, статусы	Приглашение с обязательной вилкой; статусы sent → viewed → accepted / declined / withdrawn
Публикация вакансий, отклики	Создание и редактирование вакансий, отклики отсортированы по соответствию
Контакты скрыты до принятия или отклика	Единая точка правила `invitations/access.py` ; карточки обезличены (псевдоним)

Подбор и правила видимости

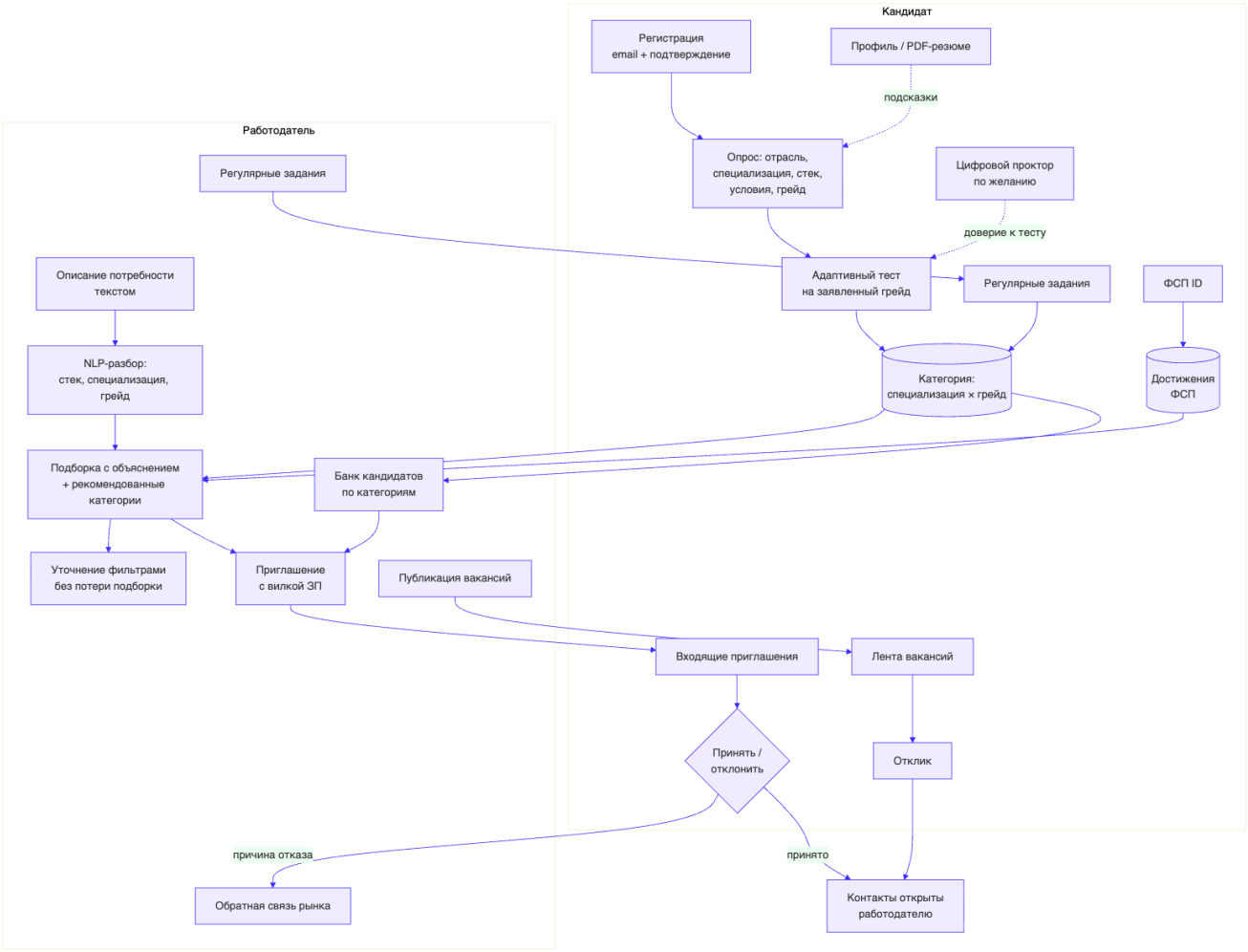
Требование	Реализация
Выдача по категориям, присвоенным тестом, а не по резюме	В поиск попадают только кандидаты с подтверждённым грейдом; резюме на категорию не влияет
Ранжирование по силе подтверждённого профиля и ФСП	Сила профиля = 0.45·тест + 0.40·ФСП + 0.15·регулярные задания (формула v7)
Объяснимость	Причины у каждого кандидата: что подтверждено тестом, что только заявлено, чего не хватает
Уточнение без потери подборки	Версии фильтров (`search_versions`), сброс уточнений одной кнопкой
Зарплата «от–до» в вакансии и приглашении	Валидация API (422) и CHECK-ограничение в БД
Регистрация по email с подтверждением	Keycloak (verifyEmail), письма — через SMTP (в локальном стенде — Mailpit)

Дополнительный функционал (критерий 6): отклик на вакансии и публикация вакансий, регулярные задания, концепция защиты от фиктивных вакансий и интеграции с ATS (реализованы квоты, репутация компании и webhook), адаптивная вёрстка для мобильных устройств, цифровой проктор по желанию кандидата.

Архитектура решения

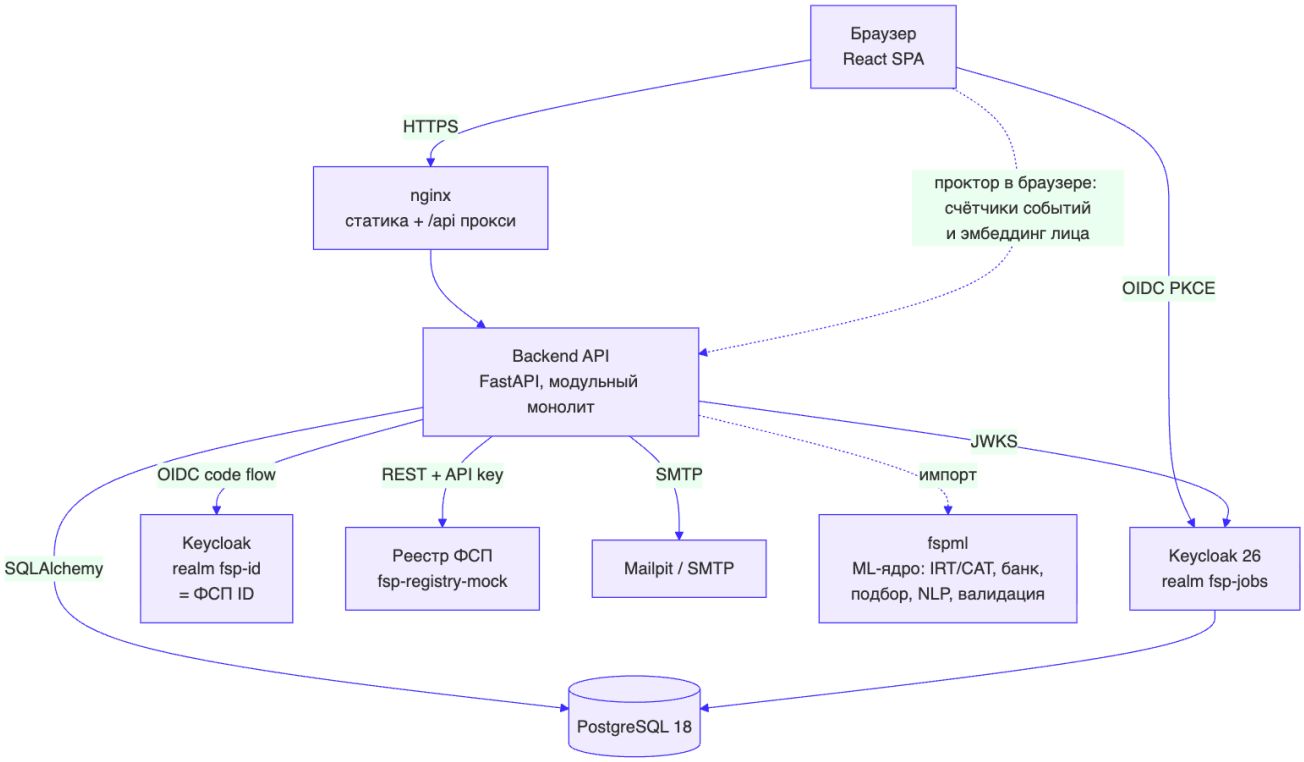
1. Функциональная архитектура

Платформа соединяет две роли напрямую, без площадок-посредников. Смысловой центр — **обратная механика**: категорию кандидата определяет тест, работодатель сам находит категорию и выходит на контакт.



Сценарий ТЗ	Где реализован
Регистрация по email с подтверждением	Keycloak realm `fsp-jobs` (verifyEmail), письма — SMTP (Mailpit в демо)
Профиль вручную или из PDF, стандартизированный PDF-профиль	`candidates` (`/api/candidate/profile` , `/resume` , `/profile.pdf`), `fspml.nlp.resume`
Опрос → грейд → тест → категория	`candidates/survey` , `assessment` , `fspml.assessment`
Регулярные задания	`tasks`
Тест на грейд выше/ниже с ограничением частоты	`fspml.assessment.grade_policy`
Приглашения: принять/отклонить, статусы	`invitations`
Вакансии и отклики	`vacancies`
Приватность, согласия, ФСП ID	`users` (согласия 152-ФЗ), `candidates/privacy` , `fsp`
Компания, потребность, подборка с обоснованием, банк с фильтрами	`employers` , `fspml.matching` , `fspml.nlp.vacancy`
Контакты скрыты до принятия/отклика	`invitations/access.py` (единая точка правила)

2. Компонентная архитектура



Компонент	Технологии	Ответственность
`frontend/`	React 19, TypeScript 7, Vite 8, Tailwind CSS 4, TanStack Query 5, React Router 7, keycloak-js 26; проктор — Human 3.3 (TF.js) и MediaPipe Tasks Vision 1.1	Два кабинета, публичные страницы механик, цифровой проктор в браузере; адаптивная вёрстка. Образ: сборка на Node.js 24, раздача — nginx 1.30
`backend/`	Python 3.14 (совместим с 3.12+), FastAPI 0.142, Uvicorn, SQLAlchemy 2.1, Alembic 1.20, Pydantic 2.13, PyJWT, httpx, fpdf2, cryptography	API бизнес-логики, правила доступа, оркестрация ML, сверка лица 1:1 (`BiometricProvider`)
`ml/` (`fspml`)	NumPy 2.5, SciPy 1.18, scikit-learn 1.9, RapidFuzz 3.14, pdfplumber 0.11, pandas 3.0	Тестирование (IRT/CAT, 303 семейства заданий), подбор, разбор текстов, валидация; устанавливается в образ `backend` как пакет
`fsp-registry-mock/`	FastAPI	Имитация реестра ФСП с предложенной схемой данных
`keycloak/`	Keycloak 26.8 (realm-export)	Аутентификация, подтверждение email, ФСП ID (демо-realm)
PostgreSQL 18	—	Реляционное хранилище (19 таблиц) + БД Keycloak
Mailpit 1.31	—	Перехват писем в локальном стенде (подтверждение email, уведомления)

Контейнеры (`docker-compose.yml`)

Шесть сервисов, весь стек — одной командой `make up` . Лимиты памяти заданы в compose; фактически стек занимает ≈ 2 ГБ RAM, основной потребитель — Keycloak. Только CPU, GPU не нужен.

Сервис	Образ	Порт на хосте	Лимит RAM	Назначение
`frontend`	сборка `frontend/Dockerfile` (Node.js 24 → nginx 1.30)	18088	64 МБ	SPA, модели проктора, прокси `/api` → `backend`
`backend`	сборка `backend/Dockerfile` (Python 3.14, uv)	18000	1 ГБ	API + ML-ядро; при старте — миграции Alembic и демо-данные
`postgres`	`postgres:18-alpine`	— (внутренняя сеть)	512 МБ	БД платформы `fsp_jobs` и БД Keycloak
`keycloak`	`quay.io/keycloak/keycloak:26.8`	18080	1 ГБ	realm-ы `fsp-jobs` (платформа) и `fsp-id` (имитация ФСП ID), импорт при старте
`mailpit`	`axllent/mailpit:v1.31`	18025	64 МБ	Письма подтверждения email и уведомления
`fsp-registry`	сборка `fsp-registry-mock/Dockerfile` (Python 3.14)	18090	128 МБ	Имитация реестра достижений ФСП

Модули бэкенда (модульный монолит)

Каждый модуль — `models.py` (таблицы), `router.py` (API), при необходимости `service.py` . Модули связаны только через явные импорты сервисных функций; бизнес-правила вынесены в одну точку (например, видимость контактов — `invitations/access.py` , переходы грейда — `fspml.assessment.grade_policy`).

Модуль	Таблицы	Назначение
`users`	`users` , `consents` , `audit_log`	Пользователь (sub из IdP), роль, согласия 152-ФЗ с версиями, журнал доступа к ПДн

Модуль	Таблицы	Назначение
`catalog`	—	Справочники из `fspml/data/catalog.yaml`
`candidates`	`candidate_profiles`	Профиль, опрос, разбор резюме, PDF, приватность
`assessment`	`attempts`, `attempt_items`, `family_stats`	Попытки теста, выданные задания (без текста — HMAC-seed), мониторинг утечек и перекалибровка
`employers`	`companies`, `searches`, `search_versions`, `shortlist`, `fsp_match_notices`	Компания, подборки с версиями фильтров, банк кандидатов, подписка на новых участников ФСП
`invitations`	`invitations`	Приглашения, статусы, квоты, обратная связь рынка, webhook ATS
`vacancies`	`vacancies`, `applications`	Вакансии и отклики
`tasks`	`regular_tasks`, `task_assignments`	Регулярные задания
`fsp`	(поля профиля)	Привязка ФСП ID, синхронизация реестра (фоновая задача)
`biometrics`	`biometric_templates`, `attempt_snapshots`	Цифровой проктор: эталон лица (AES-256-GCM, `local_demo`; в production — ЕБС), кадр с теста — работодателю только после принятого приглашения или отклика и при согласии (`/api/employer/candidates/{id}/test-photo`)

Принципы

- **ML-ядро без инфраструктуры.** `fspml` не знает о БД и вебе: чистые функции и dataclass-ы. Его тестируют (1030 автотестов) и валидируют (`make validate`) независимо; бэкенд — тонкий слой.
- **Без внешних сервисов.** Весь стек поднимается локально в Docker Compose, ключи и облако не нужны; персональные данные не покидают контур (152-ФЗ). Разбор текстов — словарь технологий + нечёткий поиск (RapidFuzz), текст PDF — pdfplumber, семантическая близость — char-n-gram TF-IDF, тесты — детерминированные генераторы, оценка кейса Middle+ — по рубрике, проктор работает в браузере кандидата.
- **Состояние теста — журнал ответов.** Попытка восстанавливается из `attempt_items`; текст задания не хранится, а воспроизводится по HMAC(секрет, попытка, позиция) — для апелляций и аудита.
- **Масштабирование.** API без состояния (горизонтально), ранжирование — SQL-фильтр по категории (сотни–тысячи строк) + векторизованный скоринг; путь роста — pgvector/ANN для семантики и материализованные агрегаты категорий.
- **Ресурсы.** Весь стек ≈ 1.5–2 ГБ RAM (Keycloak — основной потребитель), CPU-only. Подходит недорогой VPS 2–4 vCPU / 4–8 ГБ.

3. Безопасность и 152-ФЗ

Требование	Реализация
Согласие на обработку и публикацию	Обязательное согласие при выборе роли; версия и время хранятся (`consents`); отзыв скрывает профиль (`hidden`)
Разграничение прав по ролям	Роль в БД, зависимости `require_role`; тесты на 401/403/404 (`backend/tests/test_flow.py`)
Защита ПДн	Обезличенные карточки (псевдоним), контакты только после принятия/отклика, журнал раскрытия (`audit_log`), скрытие от отраслей/компаний
Пароли и токены	Пароли хранит только Keycloak (PBKDF2), бэкенд проверяет подпись JWT по JWKS, PKCE для SPA, refresh — в Keycloak-сессии
Честность теста	Ответы только на сервере, время ответа измеряет сервер, полноэкранный режим, фиксация смены вкладки, второго монитора, RDP/ВМ и автоматизации браузера; брошенная попытка = неудачная; цифровой проктор по желанию — видео не записывается, на сервер уходят счётчики событий и эмбеддинг лица
Минимизация	Ожидания по ЗП не раскрываются работодателю — только индикатор совместимости вилки; процентили ожиданий по категории показываются, если в ней не меньше 5 кандидатов
Скрытие от отраслей и компаний	Одна проверка `excluded_for` для подборки, банка, карточки, PDF, кадра с теста и приглашения; отрасль берётся у компании, а не у подборки
Вебхуки ATS	Только http(s) на публичные адреса (запросы во внутреннюю сеть стека отклоняются, редиректы не выполняются); тело подписано HMAC-SHA256
Конфигурация	С `ENVIRONMENT=production` бэкенд не стартует, если включён `AUTH_MODE=dev` или оставлены секреты из `.env.example`

Механика тестирования и категоризации

> Код: `ml/fspml/assessment/` (движок), `backend/app/modules/assessment/` (API). Результаты проверки — validation.md.

1. Задача и требования ТЗ

Категория кандидата = **специализация × грейд**. Специализацию кандидат выбирает в обязательном опросе, грейд заявляет сам, а подтверждает — тестом. Резюме на категорию не влияет.

Требование ТЗ	Решение
Единый набор заданий не подходит — задания разойдутся	Задание — **шаблон-генератор**: у каждого кандидата свой вариант; утёкший вариант бесполезен
Генерация нейросетью не гарантирует сопоставимую сложность	Сложность — свойство **шаблона** (семейства), варианты изоморфны; правильный ответ вычисляется, а не генерируется
Стабильный грейд при повторе	Классификационный адаптивный тест (CAT), правила смены грейда, проверка тест–ретест в валидации
Дискриминативность	Модель IRT 3PL, отбор по информации Фишера, мониторинг точечно-бисериальной корреляции
Тест на уровень ниже/выше, ограничение частоты	Машина состояний грейда (`grade_policy.py`)
Показать, как из вакансий формируются тесты	NLP-разбор вакансии → blueprint (доли навыков) → CAT; страница «Тест из вакансии» и `POST /api/assessment/preview`

2. Банк: параметризованные семейства заданий

Семейство (item family, подход *automatic item generation* / *item cloning*) — функция `rng → экземпляр`. Она рандомизирует параметры (числа, данные таблиц, имена, фрагменты кода, мутации) и **вычисляет правильный ответ**: Python-сниппеты исполняются, SQL выполняется в in-memory SQLite, алгоритмические задачи решаются эталонной реализацией. Ответы для JavaScript, Java, Go, C#, PHP и Vue считает Python-модель семантики языка; код заданий на Java, Go, C# и PHP — полная программа, в CI её исполняют настоящие JVM, Go, .NET и PHP, JavaScript — Node.js.

```
@family("sql.left_join", "sql", 3, "LEFT JOIN и строки без пары")
def _(rng):
    users = ...; orders = ...                    # случайные данные
    q = rng.choice([...три вида запроса...])
    ans = run_sql(setup, q)[0][0]                # ответ — результат реального запроса
    return number(prompt, ans, code=q, code_lang="sql")
```

Состав банка (`ml/fspml/assessment/bank/`): **303 семейства** по 31 навыку (в том числе направления DS: компьютерное зрение, NLP, рекомендательные системы, временные ряды; языки и фреймворки стека: Python, JavaScript, TypeScript, Java, Go, C#, PHP, React, Vue) и 7 специализациям, уровни сложности 1–5. Форматы — ввод числа/строки (предпочтительно: угадывание ≈ 0, утёкший ключ не подходит к новому варианту), выбор одного/нескольких вариантов с дистракторами из типичных ошибок, упорядочивание. Медианно семейство даёт 174 уникальных варианта из 200 генераций.

Автопроверки в CI (`ml/tests/test_bank.py` , 917 проверок): правильный ответ принимается проверкой; неверный — нет; варианты ответа не дублируются; экземпляр детерминирован по seed; у вычислимых семейств ответ меняется между экземплярами; у каждого навыка есть семейства ≥ 3 уровней; вывод заданий совпадает с Node.js, JVM, Go, .NET и PHP.

3. Модель измерения: IRT 3PL

```
P(верно | θ) = c + (1 - c) / (1 + exp(-a·(θ - b)))
```

- **b** — сложность семейства. Начальное значение — по экспертному уровню: L1…L5 → –1.75, –0.75, 0.25, 1.15, 2.0 (согласовано с порогами грейдов).
- **a** — дискриминативность, по умолчанию 1.3.
- **c** — угадывание, **фиксируется форматом** (1/k для выбора из k вариантов, 0 для ввода) и не оценивается — модель устойчива при малом числе ответов.
- **θ** — способность кандидата на шкале специализации (средний кандидат — 0). Оценка **EAP** на сетке [–4, 4] с приором N(0, 1): устойчива при всех верных/неверных ответах и сразу даёт P(θ > порог). **Достижения ФСП сдвигают приор:** N(0.8 × сила ФСП, 1), где сила ФСП — та же, что в подборке (0..1, noisy-OR по уровню, результату, релевантности дисциплины и давности). Достижения приходят из реестра, а не со слов кандидата, — это независимое свидетельство практического уровня; решение всё равно принимают ответы (сильный приор при провальных ответах — «не подтверждено», тест `test_fsp_prior_shifts_start_but_answers_decide`). Без истории ФСП приор прежний, скрытые кандидатом достижения не используются. Сила фиксируется на старте попытки (`attempts.fsp_prior`), чтобы привязка ФСП посреди теста не меняла уже идущую оценку.

Пороги грейдов (`catalog.yaml`): Intern –2.25 · Junior –1.25 · Middle –0.25 · Senior 0.75 · Lead 1.6.

4. Классификационный адаптивный тест (CAT)

Цель теста — не измерить θ «вообще», а **решить, выше ли способность порога заявленного грейда**. Отсюда алгоритм (`cat.py`):

1. **Балансировка содержания.** Следующий навык — с наибольшим дефицитом относительно персонального состава теста (§4.1). Когда до минимальной длины остаётся столько заданий, сколько не хватает до покрытия, выбор ограничивается навыками, у которых ещё меньше 2 заданий.
2. **Точка нацеливания.** Первые 3 задания — в пороге заявленного грейда; далее — в том из порогов {заявленный, следующий}, решение по которому сейчас наиболее неопределённо (P ближе к 0.5). Так тест параллельно собирает данные для предложения повысить грейд.
3. **Выбор задания** — максимум информации Фишера в точке нацеливания, случайно из топ-4 (*randomesque*) — контроль экспозиции; есть потолок частоты показа семейства по данным за 30 дней. - **Без повторов внутри попытки:** семейство не выдаётся дважды; семейства одной *группы* (один и тот же вопрос в разной форме: `GROUP BY` в SQL и `groupby` в pandas, размножение строк при `JOIN` и `merge`, A/B-тест в статистике и в метриках) — тоже. - **Без повторов между попытками:** при старте попытки в неё записываются семейства, которые кандидат видел по этой специализации за 180 дней (`attempts.seen_families`). Внутри навыка они выбираются только тогда, когда новых не осталось, — тест не обрывается. Так тест на ступень ниже после неудачи, повышение и ретест идут на *параллельной форме*, а не на тех же шаблонах с другими числами. Эффект в валидации: доля повторов в следующей попытке онбординга снизилась с 54% до 15%, на ретесте — с 62% до 28% при той же точности грейда (70,4%) (validation.md).
4. **Остановка**: не меньше 10 заданий, каждый навык состава получил не меньше 2 заданий (иначе его подоценка — «нет данных», а подтверждённые тестом навыки — главный сигнал подбора) и $P(\theta > \text{порог}) \geq 0.85$ или ≤ 0.15; не больше 25 заданий. В среднем — 16 заданий за попытку, 29 за онбординг. Порог остановки 0.85, а не 0.95: решение принимается при $P \geq 0.6$, и задания сверх уверенности 0.85 почти никогда не меняют итог (перебор и правило выбора — validation.md, «Правило остановки»).
5. **Решение**: подтверждён при $P(\theta > \text{порог}) \geq$ **0.6** (первичное подтверждение) или $\geq$ **0.70** (повышение); предложение пройти тест выше — при $P(\theta > \text{порог следующего}) \geq$ **0.5**. Пороги выбраны по валидации (validation.md): прежние 0.75 / 0.8 давали в основном *занижение* сильных кандидатов (30.7% против 2.2% завышения); новые сдвигают баланс при завышении $\leq 5\%$. Все цифры — среднее по 5 сидам, у каждого кандидата свой поток случайных чисел. Проверенная и отклонённая альтернатива — приор по заявленному грейду: ещё +3 п.п. точности, но больше ложных повышений у тех, кто завысил заявку, т. е. поощрение накрутки.
6. **Подоценки навыков** — иерархическая EAP: приор навыка $N(\theta$ без этого навыка, $0.6^2)$. При 2–4 заданиях на навык оценка сжимается к общему уровню, поэтому статус («подтверждено / частично / не подтверждено / нет данных») не скачет при повторе.

4.1 Состав теста: ядро специализации + модули стека

Было. Состав теста был общим для специализации, а заявленный стек выбирал только язык backend-теста — первый совпавший из [python, javascript], а при отсутствии совпадения подставлялся Python. В итоге задания выходили за пределы профиля: Java-, Go-, C#-, PHP-разработчики (по архетипам рынка — ~60% backend) получали 20% теста по Python, Vue- и Angular-разработчики — задания по React, Linux-администраторы — по Kubernetes, QA на Java — по Python. В среднем по архетипам вне заявленного стека оказывалось 15–37% теста (backend 37%, DS 35%). Такие задания измеряют не грейд, а знакомство с чужим стеком, и систематически занижают оценку; к тому же 5%-навыки получали по одному заданию и оставались «нет данных».

Стало (`catalog.yaml` → `core` / `modules` , `assessment/blueprint.py`):

- **Ядро** — фундамент специализации, у всех кандидатов категории (сопоставимость грейдов, языконезависимые якоря шкалы). Отобрано по покрытию архетипов стека (`stack_archetypes.yaml`) плюс общепринятый фундамент: Backend — алгоритмы, SQL, HTTP, архитектура; Frontend — JavaScript, HTML/CSS, HTTP, алгоритмы; Mobile — мобильные платформы, HTTP, алгоритмы, архитектура; DS — ML, статистика, Python; QA — тест-дизайн, HTTP; DevOps — Linux, Docker, CI/CD, сети; аналитик — требования, SQL.
- **Модули** — навыки, которые подтверждают заявленные технологии (Django и FastAPI → Python, Spring → Java, Gin → Go, .NET → C#, Laravel → PHP, React → React, Vue → Vue, Appium → мобильные платформы): до 3 модулей, 20% теста на модуль, не больше 40% на все (ядро — не меньше 60%); внутри — пропорционально √числа технологий (язык с фреймворками весит больше Git).
- **Технология без заданий** (например, Rust, Ruby, Scala, Angular) ничем не подменяется: тест опирается на ядро, в профиле она остаётся «заявленной». Кандидат видит свой состав до старта: ядро, модули с технологиями-источниками и список технологий без заданий (`GET /api/assessment/eligibility` → `blueprint`).

Направления внутри специализации. У DS есть направления — табличные данные, компьютерное зрение, NLP, рекомендательные системы, временные ряды (`catalog.yaml` → `directions`). Кандидат выбирает до двух в опросе. Навык направления входит в тест модулем с приоритетом над стеком (весит как две заявленные технологии): CV-инженер получает задания по IoU, NMS, свёрткам и валидации на видео, даже если не перечислил OpenCV. Лимиты модулей прежние, ядро — не меньше 60%, поэтому грейды в категории остаются сопоставимыми. Механизм общий и может быть включён для других специализаций.

Тест подтверждает то, что кандидат заявил, и не проверяет того, чего он не заявлял; накрутка стека при этом не выгодна — заявленная технология превращается в модуль и проверяется.

Проверка (validation.md §2.3): на 600 кандидатах с реалистичными стеками, когда стек не влияет на ответы, новый состав сопоставим с прежним по точности (67.2% против 67.8%, в пределах шума) и стабильнее при повторе (72.0% против 67.7%); когда влияет (способность в чужом стеке на 1 SD ниже общей), прежний состав занижает грейд у затронутых кандидатов в 36.7% случаев (смещение θ −0.14), новый — в 27.1% (−0.02). Доля заявленных технологий, по которым у работодателя есть данные теста, растёт с 77% до 94%, и для подбора это заметно: подоценки теста при прежнем составе ухудшали выдачу (−0.023 NDCG@10), при новом — улучшают (+0.067, validation.md §4). Цена — +0.4 задания на попытку.

5. Правила грейда (машина состояний)

Ситуация	Правило
Онбординг (грейд ещё не подтверждён)	Тест на любой грейд. После неудачи на грейде g: ниже g — сразу, g и выше — через 14 дней
Первое подтверждение	Не считается сменой; сразу доступно повышение на одну ступень
Повышение	Только на одну ступень; смена грейда не чаще раза в **90 дней**
Неудача на более высоком уровне	Текущий грейд сохраняется — **грейд не понижается** по результату теста
Повтор на свой грейд (обновить результат)	Раз в 14 дней

Ситуация	Правило
Брошенная попытка	Засчитывается как неудачная (защита от «перезапуска до удачного набора»)
Смена специализации	Новая категория: прежний результат не переносится

6. Тест из описания вакансии

`fspml.nlp.vacancy.parse_need` выделяет технологии с важностью (обязательные / «будет плюсом» по секциям текста), специализацию и грейд. Технологии сопоставлены навыкам теста (`skills.yaml` : PostgreSQL → `sql` , FastAPI → `python` + `http` , …). **Blueprint** вакансии = 40% ядра специализации (якоря шкалы и сопоставимость с категорией) + 60% навыков из вакансии пропорционально важности; язык берётся из текста вакансии (Java-вакансия не получает Python «по умолчанию»). Дальше работает тот же CAT. Страница `/test-demo` показывает двух кандидатов одинаковой силы: составы навыков совпадают, формулировки — нет. Blueprint используется в скрининге по вакансии и в регулярных заданиях работодателя.

7. Почему механика устойчива к распространению заданий

1. **Утечка экземпляра бесполезна.** У другого кандидата другие числа, данные и код — и другой ответ. Хранить «ответы к тесту» невозможно: их пространство огромно.
2. **Адаптивность.** Разные кандидаты получают разные семейства: максимальная экспозиция семейства в симуляции — ~40%, медианная — ~8%.
3. **Контроль экспозиции.** Randomesque-выбор и потолок частоты показа.
4. **Знание шаблона ≈ подготовка по теме.** Утечка *решения шаблона* даёт частичный выигрыш (как изучение темы), а не готовый ответ. В симуляции при утечке 50% банка ложные подтверждения грейда: фиксированный тест — **60%**, наш подход — **23%** (см. validation.md, §2.4).
5. **Мониторинг дрейфа (CUSUM).** По каждому семейству копится сумма остатков «факт – ожидание» ($S = \max(0, S + x - p - 0.12)$); при $S > 7$ семейство помечается и выводится из ротации до ревизии. В симуляции утечка, которой пользуется половина кандидатов, ловится во всех случаях медианно за ~75 ответов при ~2% ложных тревог.
6. **Сервер — источник истины.** Правильные ответы и seed не покидают сервер; вариант восстанавливается по HMAC(секрет, попытка, позиция) для апелляций; время ответа измеряет сервер: превышение рекомендуемого времени на задание фиксируется в отчёте о прохождении, но на оценку не влияет. Поведенческие сигналы (уход со страницы, копирование, вставка) влияют на результат — см. §9.
7. **Онлайн-перекалибровка.** Параметры семейств уточняются по ответам методом MML-EM (байесовская мода с экспертным приором): ошибка сложности в симуляции снижается с 0.28 до 0.23 по 17 тыс. ответов — «подобравшее» утечку семейство становится видно и по параметрам.

8. Второй этап для Middle, Senior, Lead: практический кейс

Почему. Тест знаний хорошо отделяет Intern/Junior, но грейд выше определяется самостоятельностью, работой с неопределённостью, компромиссами и архитектурой (карьерные матрицы GitLab, Dropbox). По пересмотренному мета-анализу Sackett et al. (2022) самые валидные методы отбора — структурированное интервью ($r = 0.42$), тесты профессиональных знаний (0.40) и рабочие образцы (0.33). При этом короткие вопросы «что выведет программа» легко решаются нейросетевыми помощниками — поэтому для старших грейдов нужен формат, который сложнее пройти чужим ответом.

Формат. После теста знаний — кейс уровня грейда (15–18 мин; `ml/fspml/data/scenarios.yaml` : 21 кейс, 7 специализаций × 3 грейда, у каждого параметры варианта) и **уточняющий вопрос по ответу кандидата** с таймером 2 минуты. Вопрос выбирается из заранее написанных: по самому слабому критерию ответа, а если все раскрыты — «смена требований». Неожиданное «почему так?» и изменение условия — то, что сложнее всего пройти пересказом чужого ответа (Meta AI-enabled coding, 2025; conversational exam, Barba & Stegner, 2026).

Сюжет из области кандидата. У кейса один скелет — критерии, веса, вопросы, параметры GRM — и несколько *обёрток* (skins) с разными сюжетами: заголовок, контекст и значения сюжетных переменных `{{var}}` , которые подставляются и в рубрику, и в её ключевые слова. Пример: DS Middle «модель с accuracy 97%» — отток клиентов банка (финтех), детектор фолов по видео матча (CV · спорт), прогноз травм (спорт), классификатор эскалаций в поддержке (NLP), отказ насоса по датчикам (временные ряды · промышленность), повторная госпитализация (медицина). Ловушки те же — дисбаланс классов, утечка будущего, группировка по клиенту/матчу/насосу. Для других специализаций — та же механика: спортивные версии кейсов Middle у backend («медленная история матчей игрока» вместо «списка заказов»), QA (промокод на абонемент фитнес-клуба) и аналитика (возврат билета на матч); кейсы фронтенда, мобильной разработки и DevOps от отрасли почти не зависят. Обёртка выбирается по направлению кандидата (2 балла) и отраслям из опроса (1 балл), у базовой версии свои теги; при повторе кейса берётся ещё не виденная версия. Тест `test_skins_keep_rubric_and_difficulty` проверяет, что эталонный ответ каждой обёртки оценивается так же, как базовый, а слабый — так же слабо.

Соответствие ТЗ. Кейс фиксирован и написан заранее, сложность не генерируется нейросетью: у всех версий кейса одни критерии и веса.

Оценка (`ml/fspml/assessment/scenario_grader.py`). Рубрика с описанными уровнями (BARS): 0 — не затронуто, 1 — в общих словах, 2 — ожидание грейда, 3 — сверх уровня. Для каждого критерия в рубрике заданы ключевые признаки ответа (регулярные выражения, с подстановкой сюжетных переменных обёртки). Балл критерия — по числу найденных признаков: 0 → 0, 1 → 1, 2–3 → 2, 4 и больше → 3; ответ короче 300 символов не получает «сверх уровня», ответ короче 40 символов — 0. Найденные фрагменты сохраняются как обоснование балла. Оценка детерминирована и работает без сети; результат помечается как «предварительная автоматическая оценка» (`method: keywords`), а ответ кандидата сохраняется в попытке для перепроверки. Итог: 70% критерии + 30% уточняющий вопрос → категория 0–3.

Как кейс входит в решение. Категория — политомический ответ в модели градуированных ответов (GRM, Samejima, 1969): $P(k \geq j \mid \theta) = \sigma(a(\theta - b_j))$, $a = 1.8$, $b = \text{cut} - 1.0 / \text{cut} / \text{cut} + 0.9$. Порог «соответствует уровню» совпадает с порогом грейда: кандидат ровно на пороге даёт такой ответ с вероятностью 1/2. Правдоподобие кейса умножается на апостериорное распределение θ после теста знаний, и решение принимается по общей шкале, без ручных весов. Слабый кейс при сильном тесте знаний заметно снижает $P(\theta > \text{порог})$ — именно так отсекается «прошёл квиз с подсказкой». Параметры GRM экспертные и уточняются по пилоту.

9. Надёжность прохождения

`ml/fspml/assessment/integrity.py`. Решение принимается по совокупности сигналов, правило известно кандидату до начала:

- **Уход со страницы** (`visibilitychange` + `blur` окна) дольше 2 с — с длительностью по каждому заданию. При возврате кандидат сразу видит предупреждение и число оставшихся допустимых уходов.
- **Копирование текста задания и вставка в ответ** отключены, попытки считаются.
- **Согласованность ответов** — индекс lz (Drasgow, Levine & Williams, 1985): «верно на трудных, неверно на лёгких». Это мягкий сигнал; у честных симулянтов ложных пометок меньше 6% (тест `test_lz_false_alarm_rate_on_honest_responses`).

Уровень	Условие	Последствие
void	уходы на ≥ 3 заданиях, или суммарно ≥ 60 с вне страницы, или ≥ 3 попытки вставки	попытка аннулируется сразу и считается неудачной (повтор через 14 дней)
review	единичные уходы, копирование/вставка, lz < –2	результат засчитан, работодатель видит замечания
clean	—	«пройден без замечаний»

Ограничение честно: клиентские сигналы не видят второе устройство и оверлеи. Поэтому главная защита — формат: кейс плюс уточняющий вопрос с таймером. Детекторы «текст от нейросети» не используются: они ненадёжны и дают ложные обвинения.

Полноэкранный режим и окружение (все тесты на десктопе, `frontend/src/lib/environment.ts`). Тест открывается в полноэкранном режиме по клику «Начать»; вне его задание скрыто, а время вне режима засчитывается как уход со страницы (уход на вкладку и выход из режима не суммируются — переключение вкладки само выводит из режима). Если браузер отказал в режиме (встроенные окна, политики), тест не блокируется, но работодатель видит пометку. Пометки без аннулирования:

Сигнал	Как определяется	Что ловит / ограничение
Окно уменьшено внутри полноэкранного режима	размер окна < 85% размера при входе в режим	открытые инструменты разработчика
Второй монитор	`screen.isExtended` (Chromium), событие `change`	подсказчик на втором экране; в Firefox/Safari — неизвестно
Удалённый рабочий стол / виртуальная машина	имя видеоадаптера WebGL (`UNMASKED_RENDERER`), классифицирует сервер: «Microsoft Remote Display Adapter», «Basic Render Driver», VMware, VirtualBox, Parallels, Hyper-V, QEMU	сеанс RDP Windows и ВМ; программный рендеринг не считается сигналом (бывает при отключённом ускорении); легальные корпоративные VDI — поэтому только пометка
Автоматизация браузера	`navigator.webdriver`	боты и агенты, управляющие браузером

Ограничение честно: удалённый доступ к физическому компьютеру (TeamViewer, AnyDesk, Chrome Remote Desktop) браузер не видит — для него ввод помощника неотличим от ввода кандидата. Против этого работают проктор (речь, телефон, взгляд, сверка лица) и формат кейса с уточняющим вопросом с таймером. Для production и грейдов Senior/Lead — экзаменационный браузер (Safe Exam Browser: блокирует удалённый доступ, ВМ и демонстрацию экрана; сервер проверяет его по заголовку `X-SafeExamBrowser-ConfigKeyHash`). Перспективный, но требующий пилота сигнал — модель времени ответа (van der Linden): путь «увидеть → спросить ИИ → ввести» даёт долгие паузы на лёгких заданиях с верным ответом.

Водяной знак. На карточке каждого задания — бледный код попытки и кандидата. Экземпляры заданий и так уникальны (`instance_seed`), а код делает источник фотографии очевидным: это сдерживает фотографирование заданий.

9.1 Цифровой проктор (по желанию кандидата)

Закрывает то, чего не видят браузерные сигналы: телефон, которым фотографируют экран; подсказчика рядом; зачитывание заданий вслух; подставного сдающего.

Что работает на устройстве кандидата (`frontend/src/lib/proctor/`; библиотека Human на TF.js и MediaPipe Tasks Vision; модели раздаются своим nginx из `/models/human` и `/models/mediapipe`, без внешних CDN):

Детектор	Модель / метод	Счётчик на задание
Лицо в кадре, несколько лиц	BlazeFace	`no_face_ms`, `multi_face_ms`
Телефон в кадре	два детектора COCO, класс «cell phone»: CenterNet из Human (порог 0.35) и, если он не нашёл, EfficientDet-Lite0 из MediaPipe (порог 0.4) — по всему кадру и вторым проходом по области вокруг лица со смещением вниз. Эпизод начинается с первого кадра с телефоном и заканчивается после двух кадров без него; при уверенности 0.2–порог следующий кадр берётся через 150 мс вместо 500	`phone_hits`
Взгляд вбок или вверх	поворот головы по FaceMesh (	yaw
Взгляд вниз (лист, клавиатура)	наклон головы вниз > 29° — разрешён, только счётчик	`look_down_ms`
Речь (зачитывание вслух)	VAD: энергия в полосе 300–3400 Гц выше адаптивного шумового порога на 15 дБ, фраза ≥ 1,5 с	`speech_ms`
Камера закрыта / выключена	трек `ended` / `muted`, кадр тёмный или без контраста	`cam_lost_ms`
Микрофон выключен	трек `ended` / `muted` / `enabled = false`, «цифровая тишина» ≥ 10 с	`mic_lost_ms`

Детектор	Модель / метод	Счётчик на задание
Живое лицо (на проверке перед стартом)	активная проверка: поворот головы в обе стороны (	yaw
Отпечаток лица	faceres → эмбеддинг 1024; только с качественного кадра: одно лицо ≥ 160 px, уверенность детектора ≥ 0.8, лицо смотрит в камеру (	yaw

Видео и звук не записываются и не передаются. На сервер уходят счётчики по заданию, эмбеддинг лица на старте и примерно раз в минуту (в случайный момент) и — при отдельном согласии — один кадр JPEG 320×240 с проверки перед стартом.

Цепочка доверия к личности.

1. *Владелец профиля — реальный человек:* привязка ФСП ID (OIDC) даёт бейдж «Личность подтверждена · ФСП ID»; в production — дополнительно ЕБС.
2. *Все попытки сдаёт один и тот же человек:* первая попытка с проктором создаёт эталон лица, следующие сверяются с ним **на сервере** (`backend/app/modules/biometrics/`). Клиентскому флагу «совпало» не доверяем. Сверка только 1:1 с эталоном своего профиля, поиска по базе (1:N) нет. Сходство считается той же формулой, что `human.match.similarity` (порт сверен с JS на случайных векторах). Эталон — галерея до 5 отпечатков: 3 качественных кадра с первой проверки и лучшие кадры последующих уверенных совпадений (сходство ≥ 0.6; свет, очки, причёска со временем меняются); сходство — максимум по галерее. Решение трёхзначное: ≥ `BIOMETRIC_THRESHOLD` (0.5, как в демо faceid Human) — совпало, < `BIOMETRIC_MISMATCH_THRESHOLD` (0.35) — уверенно другое лицо, между ними — серая зона, которая сигналом не считается. На старте берётся лучший из кадров проверки. Сходство каждой сверки сохраняется в `AttemptItem.proctor` — по нему пороги калибруются на пилоте. Пересоздание эталона после отзыва согласия видно работодателю: иначе подставной сдающий мог бы незаметно занять место владельца.
3. *Сдававший — тот, кто пришёл на собеседование:* кадр с проверки виден работодателю только после принятого приглашения или отклика (`GET /api/employer/candidates/{id}/test-photo` , запись в журнал доступа).

Правила (`IntegrityConfig` , консервативные: ложное срабатывание детектора не должно аннулировать честного кандидата):

Сигнал	Последствие
Лица нет в кадре дольше 20 с	засчитывается как уход со страницы (общий счётчик: 3 задания — аннулирование)
Телефон в кадре на 1–2 заданиях / на ≥ 3	review / void
Несовпадение с эталоном на старте; эпизод несовпадения во время теста (≥ 2 уверенно плохие сверки подряд)	review
Два эпизода, эпизод вместе с несовпадением на старте, или эпизод из ≥ 3 сверок	void
Один плохой кадр; сверки в серой зоне	не сигнал (поворот головы, наклон к листу, свет)
Несколько лиц; речь на ≥ 2 заданиях; взгляд вбок или вверх на ≥ 3	review
Взгляд вниз (лист с расчётами)	не сигнал: счётчик заданий виден работодателю
Камера или микрофон отключены суммарно ≥ 30 с	не нарушение: бейдж «Тест с проктором» снимается, попытка продолжается

Кандидат видит предупреждения в реальном времени и может исправить ситуацию (убрать телефон, включить микрофон). Локальное предупреждение «лицо не похоже» появляется только после двух уверенно плохих сверок подряд. После перезагрузки страницы тест не продолжается без камеры; время без неё засчитывается как отключение камеры.

Собственное видео — только на проверке перед стартом. Во время теста кандидат видит индикаторы (камера, микрофон, «вы в кадре», телефон), но не картинку: по ней легко найти слепые зоны камеры, например чтобы сфотографировать задание. Если лица нет в кадре дольше 3 с, индикатор подсвечивается.

Расчёты. Ручка и лист разрешены (взгляд вниз не замечание). В тесте есть черновик и калькулятор (`frontend/src/components/scratchpad.tsx` , разбор выражения без `eval`): содержимое не уходит на сервер, вставка извне в черновик считается попыткой вставки, а копирование из черновика в ответ разрешено.

Тяжесть замечаний (`IntegrityReport.severity` , сохраняется в профиле): `hard` — прямые признаки подмены или подсказки: несовпадение лица (старт или эпизод), телефон в кадре, посторонние в кадре на ≥ 2 заданиях, webdriver, удалённый рабочий стол или ВМ, попытки вставки; `soft` — остальные замечания (единичные уходы, копирование, второй монитор, уменьшенное окно, речь, взгляд вбок, lz); `none` — без замечаний.

Ранжирование. Доверие к результату — множитель итогового скора подбора (`TRUST` в `ml/fspml/matching/scoring.py` , подробно — matching.md): проктор без замечаний 1.0, с мягкими 0.9, с тяжёлыми 0.8; без проктора (или проктор прерван) 0.6, с замечаниями 0.5. Результат без наблюдения легко списать, поэтому тест с проктором даже с тяжёлыми замечаниями весит больше чистого теста без проктора; прерванный проктор приравнен к тесту без проктора, иначе выгодно включить проктор и выключить камеру. Кандидат знает об этом заранее (правила теста и окно согласия).

Правовая рамка (требует проверки юристом перед production).

- Эмбеддинг лица, используемый для установления личности, — биометрические персональные данные (ст. 11 152-ФЗ): отдельное письменное согласие `proctoring_biometric` с целью, составом, сроком и правом отзыва. Отказ не закрывает доступ к тесту без проктора.
- Кадр для работодателя — отдельное согласие `proctoring_photo` с явным указанием передачи; срок хранения 180 дней; отзыв удаляет кадры.
- 572-ФЗ: аутентификация по биометрии в production должна идти через ЕБС или аккредитованные системы. Поэтому эталон хранится за интерфейсом `BiometricProvider` : `local_demo` (наша БД, AES-256-GCM) — только для демонстрации; `ebs` — точка интеграции, где эталон хранит ЕБС, а мы проверяем подписанный результат.

Ограничения. Телефон вне поля зрения камеры и наушник детектор не увидит; модифицированный клиент может подделать счётчики (проктор повышает цену обмана, но не делает его невозможным — поэтому сверка лица на сервере, а формат с кейсом и уточняющим вопросом остаётся основной защитой). Порог сходства faceres нужно калибровать на пилоте с реальными веб-камерами: на мелких лицах (< 150 px) сходство разных людей завышено, поэтому отпечаток берётся только с крупного лица. Камера и микрофон доступны браузеру только по HTTPS или на localhost.

10. Ограничения и развитие

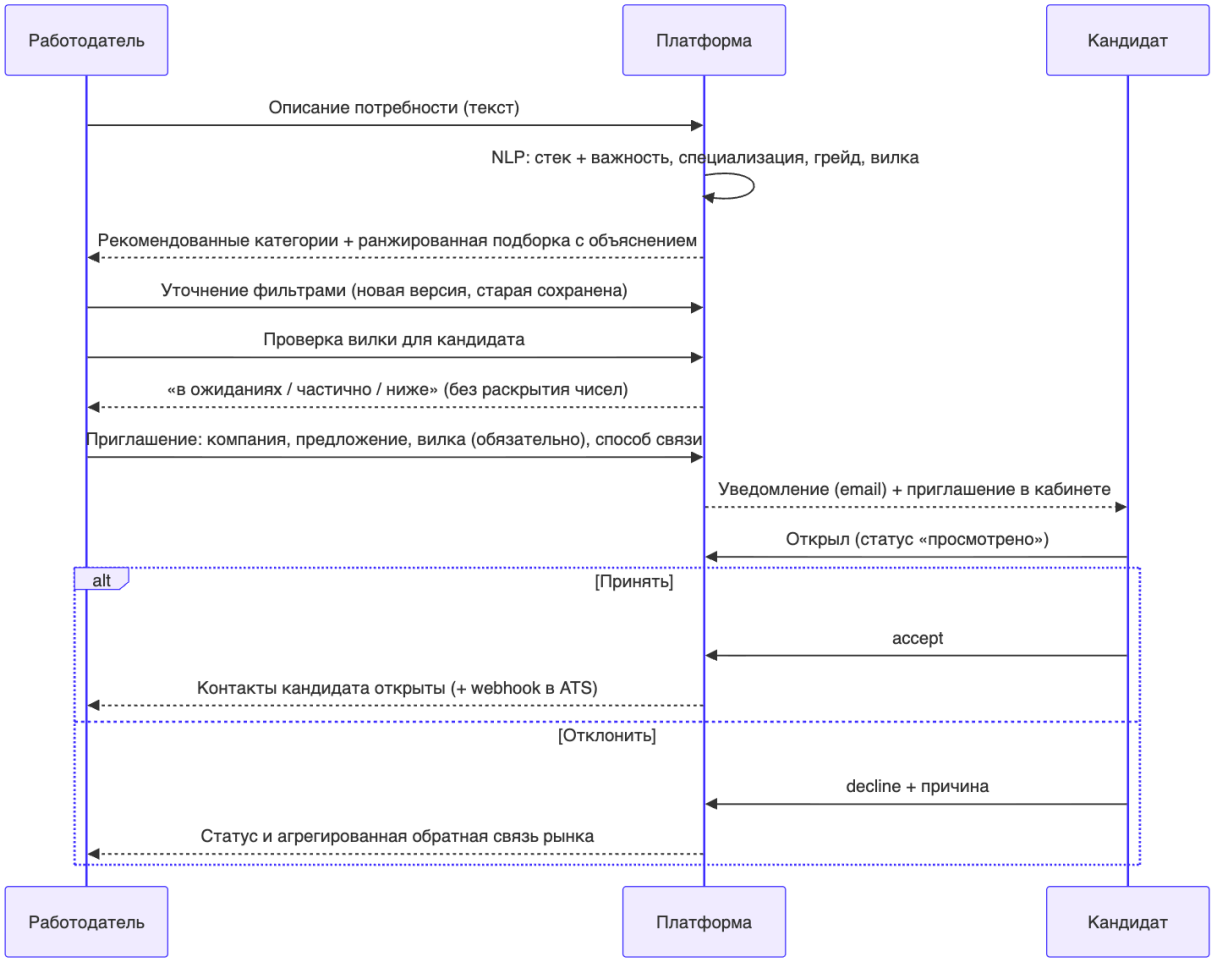
- Начальная калибровка экспертная; нужен живой пилот (протокол — pilot.md) и накопление ответов для MML-EM.
- Совпадение грейда при ретесте упирается в точность измерения на границах грейдов (~75% при 25 заданиях); для продукта это компенсирует правило «грейд не понижается».
- Концептуальные вопросы с выбором имеют небольшое число формулировок; их доля ограничена, приоритет развития — вычислимые форматы.
- Развитие: задачи с исполнением кода кандидата в песочнице как отдельный формат (результат — по тем же правилам IRT), многомерная IRT по навыкам при накоплении данных.

Механика подбора и выхода на контакт

> Код: `ml/fspml/matching/scoring.py`, `ml/fspml/nlp/`, `backend/app/modules/employers/`, `backend/app/modules/invitations/`. Результаты проверки — validation.md.

1. Принцип: инициатива у работодателя

Кандидат не откликается вслепую. Он один раз проходит опрос и тест, получает категорию — и дальше работодатели сами находят его и приходят с конкретным предложением и зарплатой. Классический отклик на вакансию тоже есть, но это дополнительный сценарий.



2. Категоризация

- **Категория** = специализация (опрос) × грейд (тест). Справочник: 7 специализаций × 5 грейдов (`catalog.yaml`).
- В поиск попадают только кандидаты с **подтверждённым** грейдом, согласием на публикацию и статусом поиска «ищу» или «открыт к предложениям».
- Резюме, заявленный стек и самооценка грейда на категорию не влияют: они подсказывают в опросе и используются как «заявленное» при ранжировании.

3. Разбор потребности (NLP, локально)

`parse_need(text, title)`: 1. Секционирование текста: «Требования / Обязательно» (вес 1.0), вступление (0.85), «Задачи» (0.8), «Будет плюсом» (0.45), «Условия» (0.2). Распознаются и строки вида «Будет плюсом: Kubernetes, gRPC», и маркеры внутри одной строки: «Обязательно: Go, PostgreSQL. Будет плюсом: Kafka», «нужен Go-разработчик, обязательно Go, будет плюсом Kafka», «опыт с Kafka будет плюсом»; «не обязательно» требованием не считается. Короткий пункт списка с технологией («- Знание Redis обязательно») — содержимое, а не заголовок. (Раньше однострочное описание целиком уходило в «будет плюсом», а такие пункты терялись: на однострочных вакансиях бенчмарка доля верно размеченных «обязательно / плюс» выросла с 36% до 62%, остальное — рассинхронизация сохранённых текстов с перегенерированной истиной.) 2. Извлечение технологий по словарю `skills.yaml` (~170 технологий с алиасами, русские и английские написания, опечатки — `rapidfuzz`). 3. Специализация — взвешенная сумма индикаторов технологий + сильный сигнал заголовка («Senior Frontend-разработчик»). 4. Грейд — маркер в заголовке → требуемый стаж («опыт от 2 лет») → маркер в тексте. 5. Вилка («250–320 тыс. ₽», «от 300 000 до 400 000 руб», «200-260k ₽») и формат работы. 6. Blueprint теста под вакансию.

Каждый шаг объясняется работодателю («Специализация Backend: заголовок вакансии», «Грейд Middle: требуемый опыт от 2 лет») и может быть исправлен вручную.

4. Ранжирование: два скора

4.1 Сила профиля — порядок *внутри категории*

Используется в банке кандидатов и как компонент выдачи. Ровно по ТЗ: выше те, у кого лучше тест и есть подтверждённые достижения ФСП.

`profile_strength = 0.45·test + 0.40·fsp + 0.15·tasks` (v7; в v5 было 0.60 / 0.25 / 0.15)

- **test** = Φ((θ – порог грейда) / 0.8) – штраф за неточность (SE > 0.35);
- **fsp** = 1 – Π(1 – s_i) по достижениям (noisy-OR — насыщается: десять участий не равны победе), где s_i = уровень (междунар. 1.0 / всеросс. 0.75 / регион. 0.45 / прочие 0.25) × результат (победитель 1.0 / призёр 0.8 / финалист 0.6 / участник 0.3) × релевантность дисциплины специализации × 0.5^(лет/2);
- **tasks** — регулярные задания с затуханием (полураспад 60 дней).

Тест остаётся главным в силе профиля (0.45 ≥ 0.40), но ФСП больше не «довесок»: доля ФСП в итоговом скоре выдачи — 0.35 × 0.40 = **14%** вместо 0.30 × 0.25 = 7.5% (§4.5).

Нет истории ФСП → fsp = 0, без штрафа; работодатель видит нейтральное «ФСП ID не привязан». Скрытые кандидатом достижения не используются и в ранжировании (иначе косвенная утечка).

4.2 Соответствие потребности — выдача подборки

Версия в продукте — v7 (`rank(..., version=7)`, `FSP_RANKER=v7`): аддитивная формула v1 ниже, в которой (1) требования вакансии весят по их **специфичности** в категории, (2) жёсткие расхождения работают как **«ворота»** — множители, а не слагаемые (v5), (3) отсутствие основного языка/фреймворка вакансии — отдельные «ворота» × 0.25 (v6), (4) ФСП весомее и засчитывается как опыт работы (v7). Обоснование и проверка — §4.4–4.5.

`match = (0.35·need_fit + 0.35·profile_strength + 0.20·conditions + 0.10·freshness) × ворота` (× 0.85 для соседнего грейда; в v5 — 0.40 / 0.30 / 0.20 / 0.10)

- **need_fit** = 0.75·покрытие требуемых технологий + 0.25·семантическая близость текста потребности и профиля (char-n-gram TF-IDF, локально; при сборке с `WITH_EMBEDDINGS=1` — модель rubert-tiny2 на CPU). Подтверждённость технологии t с навыками теста A(t):
- заявлена и навык подтверждён тестом: 0.4 + 0.6·P(θ_навыка > порог грейда потребности) → до **1.0**;
- заявлена, тест по навыку не проводился: ≈ 0.6; заявлена, но тест опроверг: 0.4;
- не заявлена, но навык силён: ≤ 0.25 (общий потенциал);
- технология без навыка в тесте (например, Java): заявлена — 0.6.
- **conditions** = 0.6·вилка + 0.3·формат + 0.1·отрасль — это прокси готовности *принять* приглашение (цель механики — принятые приглашения, а не просмотры).
- **freshness** = exp(–дней с последней активности / 60).
- **Специфичность требования** t в пуле категории: $w_t = \sqrt{(-\ln((df_t + 1)/(N + 2)))}$, где df_t — сколько из N кандидатов заявили t (как IDF в поиске). Покрытие = Σ важность$_t$·w_t·подтверждённость$_t$ / Σ важность$_t$·w_t. Git, Docker, REST есть почти у всех (w ≈ 0.8) и не отличают кандидатов; Go, PHP, Vue, Appium определяют стек вакансии (w ≈ 1.3–1.6).
- **Ворота** (множители 0..1): обязательные требования (важность ≥ 0.6) закрыты по весу меньше чем наполовину — × (0.5 + покрытие обязательного); формат «офис против удалёнки» — × 0.45, через гибрид — × 0.85; ожидания выше вилки больше чем на 15% — линейно до × 0.25. Решение рекрутера устроено как «И»: Java-тимлид без Go не становится релевантным Go-вакансии за счёт сильного теста и Docker с Kafka.
- **Доверие к результату теста** — тоже множитель (`TRUST`). Главное — был ли проктор: результат без наблюдения легко списать, поэтому проктор даже с замечаниями весит больше чистого теста без него. Внутри яруса — по тяжести замечаний попытки, подтвердившей грейд (assessment.md): hard — признаки подмены или подсказки, soft — остальные.

Попытка	×
Проктор, без замечаний	1.00
Проктор, мягкие замечания	0.90
Проктор, тяжёлые замечания	0.80
Без проктора или проктор прерван	0.60
Без проктора, с замечаниями	0.50

При одинаковом соответствии проктор с тяжёлыми замечаниями стоит выше чистого теста без проктора; кандидат без проктора обгоняет, только если подходит в ~1.35 раза лучше (тесты `test_trust_proctor_with_remarks_above_plain`, `test_much_better_fit_without_proctor_still_wins`). На бенчмарках множитель не проверяется: у синтетических кандидатов проктора нет, множитель у всех одинаков и порядок не меняет. - **Опыт работы** — множитель относительно требуемого стажа вакансии (или типичного для грейда): засчитывается релевантный опыт и 30% прочего IT-опыта; нехватка — линейно до × 0.5, превышение — до × 1.06, опыт не указан — × 0.9, от года в отрасли вакансии — × 1.04. **v7: достижения ФСП засчитываются как опыт** — до `FSP_YEARS_MAX` = 1 года × сила ФСП, и только в пределах нехватки (бонус за превышение даёт лишь реальный стаж). Свежий призёр всероссийских соревнований (сила ≈ 0.5) закрывает типичное для Junior требование «от полугода»; на Middle (2 года) — частично; на Senior почти не влияет. В карточке — «Достижения ФСП засчитаны как практический опыт (≈ N мес.)». - Жёсткие исключения: «не ищу», отрасли/компании, скрытые кандидатом, отозванное согласие, грейд дальше ±1. - **Вакансии и отклики** оцениваются той же формулой: соответствие кандидата вакансии в его ленте и в списке откликов у работодателя считается по v7 со специфичностью требований по пулу специализации, воротами, доверием к тесту и требуемым стажем вакансии («Опыт от, лет»).

4.3 Объяснимость

Каждый скор раскладывается на вклады компонентов (полоска в карточке) и сопровождается причинами:

- ✅ «Подтверждено тестом: Python, PostgreSQL, FastAPI»
- ➖ «Заявлено, но не проверено: Docker, Kafka»
- ✅ «Тест: 82-й перцентиль категории»
- ✅ «ФСП: призёр — всероссийские соревнования, продуктовое программирование, 2025»
- ✅/⚠️ «Ожидания по ЗП в вашей вилке» / «Вилка ниже ожиданий кандидата»
- ⚠️ «Соседняя категория: Senior»
- ❌ «Не подтверждено: Kubernetes» — чего не хватает до полного соответствия; если провалены «ворота» — «Не закрыто из обязательного: Go, gRPC» (самые специфичные пробелы первыми) и «Ожидания по ЗП выше вилки более чем на 15%»

Обоснование весов: компонент соответствия потребности — главный (40%), сила профиля (30%) реализует требование ТЗ о приоритете подтверждённых кандидатов, условия (20%) — практическую применимость. Валидация показывает, что для 7 из 10 весов изменение ±50% меняет NDCG@10 меньше чем на 0.04; наиболее чувствительны штраф соседней категории и вес условий — оба правила содержательно обоснованы.

4.4 Почему v5: разбор ошибок v1 и проверка

Разбор ошибок. На реалистичном бенчмарке (bench2, 700 кандидатов с живыми текстами резюме, 30 вакансий, слепая разметка) у v1 в топ-10 было 148 из 300 мест с оценкой асессора 0–1. Главная причина (~60% случаев) — **несовпадение основной технологии**: на «Senior Go» — Java-, Python- и Kotlin-лиды, на PHP/Laravel — junior на Java, C#, Go и Node.js, на Vue — React- и Angular-разработчики, на iOS — Android, на Java+Selenium+Appium — автоматизаторы на Python и ручные тестировщики, на CV — табличный DS. Механизм простой: в покрытии все требования весили одинаково, и Java-разработчик с Docker, Git, PostgreSQL, Kafka закрывал PHP- или Go-вакансию почти так же, как профильный (покрытие 0.41 против 0.43), а сила профиля и условия решали остальное. Вторая причина — несовместимые условия (формат, вилка), которые аддитивная формула не отсекает.

Проверка (nDCG@10, condensed; Δ и p — к v1 с тем же разбором вакансий, парный рандомизационный тест; варианты заданы до прогона, коэффициенты не подбирались: «ворота» — из v2, специфичность — стандартный IDF):

	bench2, Claude (слепо)	bench2, скрытая истина	bench1, GPT-5.6 Terra	bench1, Claude	bench1, скрытая истина
v1	0.704	0.767	0.789	0.801	0.690
v5	**0.792** (+0.088, p < 0.001)	0.729 (–0.038)	**0.803** (+0.013, p = 0.07)	**0.843** (+0.042, p = 0.002)	0.700 (+0.010)
v5 ⊕ кросс-энкодер	**0.813** (+0.108, p < 0.001)	0.750 (–0.017)	0.796 (+0.006)	0.813 (+0.012)	0.685 (–0.005)
для сравнения: v1 ⊕ кросс-энкодер (прежний гибрид)	0.736 (+0.032)	0.767	—	—	—

v1 здесь — с исправленным разбором вакансий (на однострочных вакансиях bench1 это само по себе +0.010…+0.011). Выигрыш устойчив к форме веса специфичности (ln, √, с потолком, 0.5 + 0.5·ln: +0.08…+0.11 на bench2 и +0.013…+0.019 / +0.031…+0.042 на bench1); выбран √ — мягкий вариант, меньше других расходящийся со скрытой истиной. Кросс-энкодер добавляет пользу только на живых текстах (bench2) и требует нейросетевой модели, поэтому в поставке была формула v5 (сейчас — v7, §4.5). Скрытая истина bench2 с v5 расходится — и расходится там, где сама неправдоподобна: она ставит «3» Java-тимлиду без Go на вакансию Senior Go и «0» автоматизаторам Java+Selenium+Appium на вакансию QA Automation (Java), потому что генератор истины не моделирует важность основного языка. Подробности, абляции и ограничения — bench.md.

4.5 v6 и v7: основной стек и весомый ФСП

Зачем. Молодые участники соревнований ФСП часто сильны, но без коммерческого стажа — а работодатель их отсеивает, и v5 повторяла эту логику: без опыта множитель × 0.5, а ФСП давал лишь 7.5% скора. Подтверждённые реестром достижения — это практика командной разработки в сжатые сроки, которую нельзя приписать себе. v7 даёт им вес, не ломая главного: тест остаётся главным в силе профиля, а нужный стек ФСП не заменяет.

Как подобраны веса. Перебор на синтетической валидации (1000 кандидатов, 30 потребностей): `ps_fsp` ∈ {0.25…0.45} (`ps_test` = 0.85 – `ps_fsp`), `profile` ∈ {0.30, 0.35}, `FSP_YEARS_MAX` ∈ {0, 1, 1.5, 2}. Правило выбора задано до перебора: максимальная доля ФСП, при которой NDCG@10 падает не больше чем на 0.01 относительно v6, и `ps_fsp` ≤ `ps_test` . Скрытая истина генератора сама понижает кандидатов без стажа («так рассуждает работодатель»), поэтому v7 с ней сознательно расходится — порог 0.01 ограничивает цену этого расхождения.

ps_fsp / ps_test	profile	FSP_YEARS_MAX	доля ФСП	NDCG@10	Δ к v6
v5	0.30	—	7.5%	0.712	+0.015
v6 (0.25 / 0.60)	0.30	0	7.5%	0.697	—
0.35 / 0.50	0.35	1.0	12.3%	0.692	–0.004
0.40 / 0.45 (v7)	**0.35**	**1.0**	**14%**	**0.691**	**–0.006**
0.40 / 0.45	0.35	1.5	14%	0.686	–0.011
0.45 / 0.40	0.35	1.0	15.8%	0.689	–0.008 (ФСП > теста — отклонено)
0.45 / 0.40	0.35	2.0	15.8%	0.679	–0.018

Перебор выполнен до смены правила остановки теста (0.95 → 0.85), поэтому абсолютные значения NDCG отличаются от раздела «Валидация». В текущем прогоне v7 0.700, v6 0.700, v5 0.721: разница v7 с v6 в пределах 0.001, выбор весов не меняется.

Проверка на слепой разметке (bench2: 700 кандидатов с живыми текстами резюме, 30 вакансий, Claude не видит, какая система нашла пару; 18 новых пар из топ-10 v5–v7 доразмечены тем же асессором по той же рубрике): nDCG@10 v5 **0.789** → v6 **0.812** → v7 **0.817**; по скрытой истине bench2 v6 0.665, v7 0.668 (bench2-results.md). То есть на независимой разметке усиление ФСП не стоит качества подборки.

Живые вакансии (bench3: 10 реальных открытых вакансий Junior/Middle × 20 обезличенных собирательных профилей): участники ФСП без стажа поднимаются в выдаче, кандидаты с нужным стеком и опытом остаются наверху — bench3-results.md. Один кандидат в двух порядках (v5 и v7) с разложением скора — fsp-showcase.md.

5. Рекомендованные категории

Для потребности показываются целевая категория, соседние грейды и смежные специализации (если разбор дал им ≥ 15%): число кандидатов, сколько с достижениями ФСП, медиана ожиданий, доля «в вашей вилке». Это реализует сценарий LHH: работодатель сначала видит структуру рынка, затем — людей.

6. Уточнение без потери подборки

Подборка — сохраняемая сущность (`searches`). Фильтры (грейд, обязательный стек, подтверждённые тестом навыки, наличие ФСП, формат, «вилка не ниже ожиданий», «активно ищущие», без соседних грейдов) применяются поверх исходной выдачи; каждое изменение сохраняется версией (`search_versions`), исходная выдача не теряется — в интерфейсе уточнения сбрасываются одной кнопкой; показывается «N из M». Кандидатов можно закрепить в шорт-листе. Подборка пересчитывается при открытии — новые кандидаты появляются автоматически.

7. Выход на контакт

Правило ТЗ	Реализация
Зарплата обязательна («от–до», ₽) — и в вакансии, и в приглашении	Валидация API (422) + CHECK-ограничение в БД
Приглашение: описание, вилка, компания, способ связи	Поля `invitations`
Контакты — только после принятия или отклика	`invitations/access.py` ; карточки и PDF обезличены (псевдоним)
Статусы видят обе стороны	sent → viewed (кандидат открыл) → accepted / declined (с причиной) / withdrawn

Дополнительно: - **индикатор вилки** до отправки — без раскрытия ожиданий кандидата; - **квота** приглашений на компанию в неделю (по умолчанию 30) и повторное приглашение того же кандидата запрещено, пока активно предыдущее; - кандидат видит **статистику компании**: сколько приглашений за 90 дней, какая доля принимается, отвечает ли на отклики — основа репутации и анти-фрода; - **обратная связь рынка** работодателю: доли причин отказов, медиана и квартили ожиданий в категориях, где он ищет, доля отказов по зарплате; - **webhook в ATS** при принятии приглашения.

8. Классический отклик (дополнительно)

Лента вакансий кандидата отсортирована той же формулой «в обратную сторону» (кандидат относительно вакансии) и показывает, почему вакансия подходит. Отклик сразу открывает контакты работодателю; отклики работодатель видит отсортированными по соответствию, со статусами viewed → interview → rejected / hired.

Валидация решения

> Сгенерировано `make validate` (331.8 с). Все числа воспроизводимы: генераторы детерминированы по seed. Сырые результаты — `docs/validation.json`.

1. Постановка

Готовых данных и эталонной разметки в задаче нет, поэтому валидация построена на **синтетической популяции со скрытой истиной** (1000 кандидатов, 30 потребностей работодателей) плюс проверки на реальных текстах вакансий. Система никогда не видит истинных признаков: только ответы в тесте, заявленный стек, текст резюме, достижения ФСП и условия.

Чтобы валидация не была цикличной, ответы на задания симулируются по модели, *отличной* от модели оценивания:

- истинная сложность семейства отклоняется от номинальной (σ = 0.35), дискриминативность — в 1.2 раза в обе стороны;
- экземпляры одного семейства неидеально равны по сложности (σ = 0.25);
- кандидат отвечает с навыко-специфичной способностью (σ = 0.45 от общей) — тест многомерен, модель одномерна;
- угадывание в заданиях с выбором реально происходит.

Эталонная релевантность пары «потребность–кандидат» (0–3) задана по скрытым признакам *до* формулы ранжирования: реальное владение требуемыми технологиями, истинная сила относительно порога грейда (знания + практика), готовность к предложению (статус поиска, вилка, формат).

Допущения генератора (их стоит проверить на живом пилоте): 30% кандидатов «накручивают» стек, чаще — более слабые и в тех областях, где они слабее; достижения ФСП отражают практический навык, частично независимый от знаний, которые меряет тест; у ~60% кандидатов истории ФСП нет.

2. Механика тестирования

2.1 Банк заданий

- **303 параметризованных семейств** по 31 навыкам, уровни сложности 1–5;
- 65.7% семейств — с вводом ответа (угадывание ≈ 0, утёкший ключ не подходит к новому экземпляру);
- из 200 генераций семейства медианно **174 уникальных экземпляров**; ≥100 уникальных — у 71.3% семейств, <10 — у 3.3% (концептуальные вопросы с выбором; их доля в тесте ограничена blueprint).
- Каждое семейство проходит автопроверки в CI: правильный ответ вычисляется исполнением кода/SQL и принимается проверкой, варианты не дублируются, ответ меняется между экземплярами; вывод заданий на JavaScript, Java, Go, C# и PHP сверяется с настоящими рантаймами.

2.2 Точность и стабильность грейда

Онбординг: тест на заявленный грейд → при неудаче на ступень ниже → при уверенном успехе одна попытка на ступень выше. Повтор (ретест) — второй онбординг того же кандидата; как и в продукте, система помнит семейства прошлых попыток и выбирает другие, то есть ретест идёт на *параллельной форме*, а не на тех же шаблонах с другими числами.

Числа этого раздела — **среднее по 5 сидам**; у каждого кандидата свой поток случайных чисел, поэтому варианты настройки сравниваются на одних и тех же случайных числах. Разброс точности между сидами — 71.5%–73.2%.

Метрика	Значение
Совпадение с истинным грейдом	**72.3%** (±1 ступень — 99.9%)
Завышение / занижение грейда	4.2% / 23.5%
Cohen's κ с истиной	0.62
Ретест: совпадение грейда	**71.4%**
Ретест: κ / взвешенная κ	0.59 / **0.83**
Ретест: корреляция θ	0.80
Ретест: совпадение решения по заявленному грейду	76.9%
Корреляция оценки θ с истинной	0.88
Заданий за попытку / за онбординг	16.2 / 29.4
Повтор по сути (то же семейство или группа): в следующей попытке онбординга / на ретесте	9.0% / 23.9%
Точность вне пограничной зоны (	θ – порог



Интерпретация. Ошибки — только на соседний грейд и в основном *в сторону занижения*: для работодателя подтверждённый грейд надёжен (завышение 4.2%), а кандидат может пройти тест на ступень выше после кулдауна. Совпадение при ретесте ограничено точностью измерения: при SE ≈ 0.33 и ширине грейда ≈ 1.0 по шкале θ теоретический потолок совпадения для кандидатов на границе грейдов — около 70%. Взвешенная к, учитывающая порядковость грейдов, — в зоне «почти полного согласия». Для продукта важнее, что **грейд не меняется вниз при повторе по правилам платформы**: повторная попытка может только подтвердить грейд или повысить его.

Выбор порога решения (P(θ > порог) для подтверждения):

Порог	Точность	Завышение	Занижение	Совпадение ретеста	Заданий
0.5	72.4%	5.4%	22.2%	69.0%	28.0
0.6	68.8%	4.4%	26.8%	67.8%	29.6
0.7	66.8%	4.2%	29.0%	69.4%	30.5
0.8	64.8%	4.0%	31.2%	69.4%	31.6

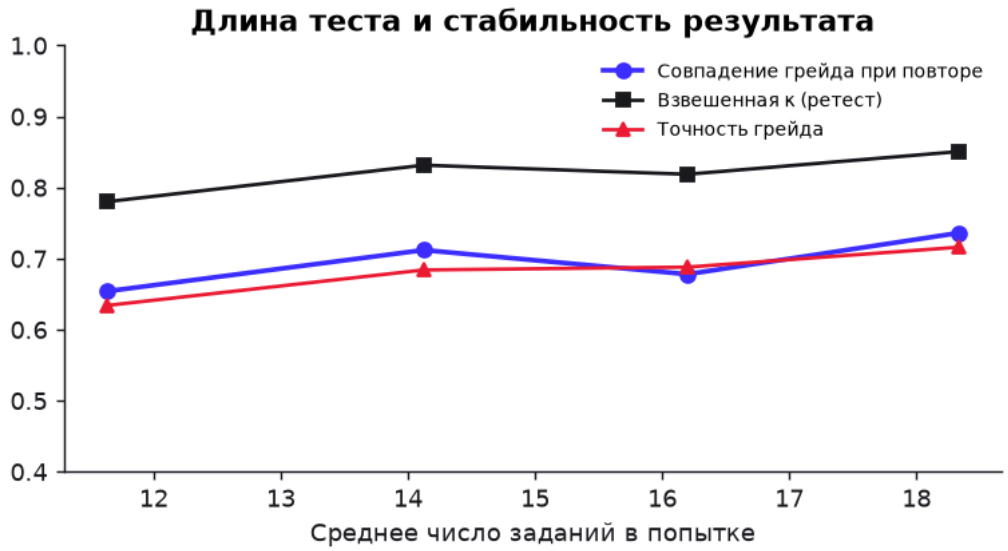
Отдельно — одна попытка теста: доля ложных подтверждений, когда кандидат заявляет грейд на ступень выше истинного, против доли подтверждений при честной заявке:

Порог	Ложные подтверждения (+1 ступень)	Верные подтверждения
0.50	9.2%	74.4%
0.60	7.0%	70.6%
0.65	7.0%	67.6%
0.70	6.8%	64.8%
0.75	6.6%	62.4%
0.80	5.8%	59.2%

Выбрано: первичное подтверждение при **0.6** — компромисс: точность итогового грейда близка к максимальной, завышение не выше 5%, а ложных подтверждений при завышенной заявке меньше, чем при 0.5 (7.0% против 9.2%). Неудача не наказывается — кандидат сразу проходит тест на ступень ниже. **Повышение** уже подтверждённого грейда — при **0.7** (ложные подтверждения 6.8%): ложное повышение дороже. Предложение пройти тест выше — при P(θ > порог следующего) ≥ 0.5.

Длина теста. Стабильность растёт с длиной теста; 25 заданий (~20–30 минут) — компромисс между точностью и временем кандидата.

Макс. заданий	Заданий в попытке	Совпадение ретеста	Взвеш. к	Точность	r(θ̂, θ)
12	11.6	65.4%	0.78	63.4%	0.85
18	14.1	71.2%	0.83	68.4%	0.88
25	16.2	67.8%	0.82	68.8%	0.88
35	18.3	73.6%	0.85	71.6%	0.88



Память о показанных заданиях. Без неё следующая попытка (например, Middle после Junior) на 48.6% состоит из уже виденных шаблонов, ретест — на 58.7%: кандидат узнаёт задачи, отличаются только числа. С памятью — 9.0% и 23.9%; остаток — навыки, где семейств меньше, чем заданий за две попытки (тогда берутся давно виденные, тест не обрывается). Точность грейда — 72.3% против 73.2% без памяти. Совпадение ретеста (71.4% против 73.1%, взвешенная к 0.83 против 0.86): повтор на тех же шаблонах завышал стабильность за счёт общей для шаблона ошибки измерения, параллельная форма — более строгая и честная оценка.

Проверенная и отклонённая гипотеза: учитывать ответы предыдущих попыток онбординга в следующей (продолжать измерение). Это сокращает тест до 23.7 заданий, но не повышает точность (65.8%) и стабильность (61.4%); по умолчанию выключено.

Точность грейда: пороги повышения, ФСП-приор и правило остановки. Главная ошибка прежней настройки — занижение сильных кандидатов, а не завышение. Пороги повышения снижены (предложить повышение — при $P(\theta >$ следующий порог$) \geq 0.5$ вместо 0.8, подтвердить — при ≥ 0.70 вместо 0.75), а достижения ФСП из реестра сдвигают априорное среднее θ на 0.8 × силу ФСП (без истории ФСП приор прежний). Само по себе это удлинило онбординг: кандидатам чаще предлагается тест на ступень выше, и каждый идёт с нуля. Длину вернула остановка при уверенности 0.85 вместо 0.95. Та же популяция и те же сиды (среднее, в скобках — min–max по сидам):

Вариант	Точно	±1	Завышение	Занижение	Точно, с ФСП	Точно, без ФСП	Взв. к ретеста	Заданий
Прежняя настройка	67.1% (66.7–67.4)	99.7%	2.2% (1.8–2.4)	30.7%	69.3%	66.0%	0.83	31.2
Только новые пороги (остановка 0.95)	71.9% (70.9–72.8)	99.9%	3.8% (3.4–4.1)	24.3%	73.5%	71.0%	0.83	35.9
Только ФСП-приор	67.6% (67.2–68.0)	99.7%	2.3% (2.0–2.5)	30.1%	70.7%	66.0%	0.83	31.2
Пороги + ФСП-приор, остановка 0.95	72.2% (70.9–73.5)	99.9%	3.9% (3.5–4.3)	23.8%	74.6%	71.0%	0.83	35.8
Текущая: пороги + ФСП-приор, остановка 0.85	72.3% (71.5–73.2)	99.9%	4.2% (3.6–4.5)	23.5%	74.9%	71.0%	0.83	29.4

Правило остановки. Тест останавливается, когда $P(\theta >$ порог$)$ выходит из интервала $(1 - c, c)$. Решение о грейде принимается при $P \geq 0.6$, поэтому задания сверх уверенности с почти не меняют итог. с выбрано перебором на сидах отбора (101–105), отличных от отчётных: минимум заданий при точности не ниже прежней – 0.5 п.п., завышении ≤ 5% и не выше прежнего + 0.5 п.п., взвешенной к ретеста не ниже прежней – 0.01 и точности у кандидатов с ФСП не ниже прежней.

c	Точно	Завышение	Точно, с ФСП	Взв. к ретеста	Заданий за онбординг
0.95	71.8%	3.6%	74.5%	0.83	35.8
0.90	71.6%	3.8%	74.0%	0.83	32.2
0.85	72.3%	4.2%	74.9%	0.83	**29.4**

Проверенная и отклонённая гипотеза: приор по заявленному грейду (самооценка как стартовая точка оценки θ) поднимает точность ещё на ~3 п.п., но увеличивает завышение у тех, кто заявил грейд выше реального (≈ 10% → 12%), — это поощряло бы завышенную заявку. ФСП-приор такой лазейки не даёт: достижения приходят из реестра, а не со слов кандидата.

2.3 Состав теста: ядро специализации + модули стека

Раньше состав теста был общим для специализации, а стек выбирал только язык (при отсутствии совпадения подставлялся Python): Java-, Go-, C#-разработчики получали пятую часть теста по чужому языку, Vue-разработчики — задания по React, Linux-администраторы — по Kubernetes. Теперь тест = **ядро специализации** (общее для всех в категории — сопоставимость грейдов) + **до трёх модулей по заявленному стеку** (доля модулей ≤ 40%); технологии без заданий ничем не подменяются. Каждый навык состава получает не меньше 2 заданий, чтобы его подоценка не оставалась «нет данных».

Эксперимент: 600 кандидатов с реалистичными стеками (архетипы рынка), backend-разработчик знает только свой язык. У 83.2% из них прежний состав содержит навыки не из реального стека. Штраф за чужой стек — насколько способность в навыке-модуле не из реального стека ниже общего уровня: 0 — стек не влияет на ответы (как в остальной валидации), 0.5–1.0 — реалистичный диапазон.

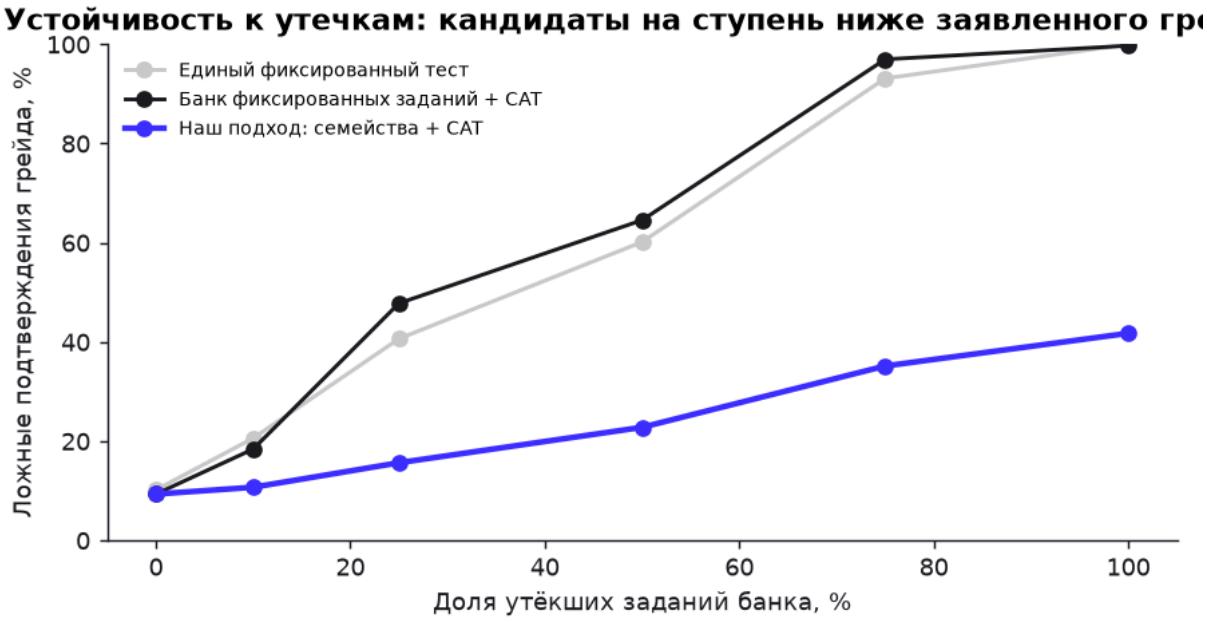
Штраф за чужой стек	Состав	Точность грейда	Точность у затронутых	Занижение у затронутых	Смещение θ у затронутых	Ретест	Заданий	Заявленных технологий с данными теста	Подтверждено: накрученные / реальные
0.0	прежний общий	68.3%	68.9%	25.7%	-0.05	71.3%	15.0	72.7%	58.4% / 64.6%
0.0	**ядро + модули**	71.2%	70.3%	24.4%	-0.08	71.5%	16.0	92.5%	74.3% / 72.6%
0.5	прежний общий	69.8%	68.7%	28.3%	-0.17	70.8%	14.7	71.8%	61.9% / 70.4%
0.5	**ядро + модули**	70.2%	69.1%	26.3%	-0.11	73.2%	15.7	92.5%	68.3% / 72.1%
1.0	прежний общий	62.3%	61.3%	36.1%	-0.27	65.3%	14.6	71.3%	65.1% / 67.9%
1.0	**ядро + модули**	67.8%	67.7%	27.7%	-0.15	67.3%	16.1	92.5%	65.7% / 72.3%

Итог. Когда стек не влияет на ответы, новый состав сопоставим с прежним (точность 71.2% против 68.3% — в пределах шума, ретест 71.5% против 71.3%). Когда влияет (штраф 1.0), прежний состав систематически занижает оценку затронутых кандидатов (смещение θ -0.27, занижение грейда 36.1%), новый — нет (-0.15, 27.7%). Доля заявленных технологий, по которым у работодателя есть данные теста, растёт с 71.3% до 92.5% — это прямой вход механики подбора («подтверждено тестом», §4), и накрутка стека чаще упирается в проверку. Цена — +1.5 задания на попытку.

2.4 Устойчивость к утечкам

Сценарий: 364 кандидатов пытаются подтвердить грейд на ступень выше истинного и имеют доступ к утечкам. Для фиксированных заданий утёкший ответ решает задание (p = 0.98). В нашем подходе утечка экземпляра бесполезна (другие параметры — другой ответ), а знакомство с шаблоном даёт лишь частичный выигрыш (как от подготовки по теме).

Доля утёкшего банка	Единый тест	Банк фикс. заданий + CAT	Наш подход
0%	10.2%	9.3%	**9.3%**
10%	20.6%	18.4%	**10.7%**
25%	40.7%	47.8%	**15.7%**
50%	60.2%	64.6%	**22.8%**
75%	93.1%	97.0%	**35.2%**
100%	100.0%	99.7%	**41.8%**



Мониторинг утечек (CUSUM). На 60 «честных» семействах по 600 ответов ложных тревог — 1.7%. Если решение шаблона утекло и им пользуется 50.0% кандидатов, семейство помечается в 100.0% случаев, медианно через 75 ответов (90-й перцентиль — 158), и выводится из ротации.

2.5 Дискриминативность и калибровка

- Использовано семейств в симуляции: 295 из 303; оценено (≥30 ответов): 182.
- Медианная точечно-бисериальная корреляция «верно ↔ истинная способность»: **0.36**; доля семейств с r < 0.2 — 9.9% (кандидаты на доработку: alg.binary_search, alg.coins, arch.queues, cicd.cache, docker.healthcheck, java.overflow, js.equality, js.typeof).
- Экспозиция: максимум 35.7% кандидатов видели одно семейство, медиана — 3.8%.

- **Онлайн-перекалибровка (MML-EM)** по 15831 ответам: средняя ошибка сложности b для 182 семейств снижается с 0.28 (экспертная разметка) до **0.24**. То есть по мере работы платформы параметры банка уточняются по данным.

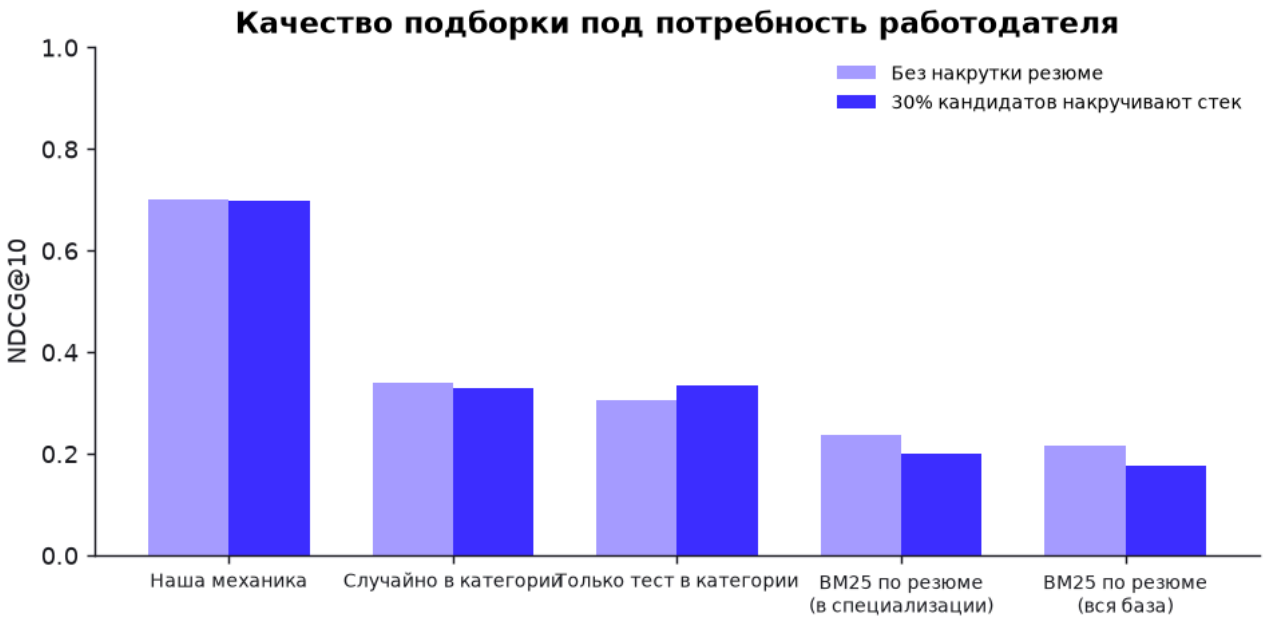
3. Категоризация

Категория = специализация (из опроса) × подтверждённый тестом грейд. Категоризированы 99.4% кандидатов; грейд категории совпадает с истинным у **69.1%**, ±1 ступень — 99.9%. Резюме на категорию не влияет.

4. Механика подбора

Тестовый набор: 30 потребностей × 1000 кандидатов, эталон — релевантность 0–3 по скрытым признакам. Метрики на топ-10: NDCG@10 (градуированная), P@10 (доля релевантных ≥ 2), MRR, ожидаемое число *принятых* приглашений среди релевантных (по модели решения кандидата).

Система	NDCG@10	P@10	MRR	Принятые@10	NDCG@10 без накрутки
Наша механика v7 (поставка)	0.700	0.673	0.933	3.95	0.702
— без ФСП	0.701	0.670	0.958	3.96	0.729
— без подоценок навыков из теста (только заявленный стек)	0.665	0.613	0.939	3.61	0.723
— без совместимости условий	0.441	0.370	0.654	1.96	0.409
Формула v6 (v5 + «ворота» основного стека)	0.700	0.660	0.958	3.89	0.729
Формула v5 (специфичность требований + «ворота»)	0.721	0.683	0.958	4.05	0.731
Исходная формула v1 (до специфичности и «ворот»)	0.618	0.593	0.883	3.35	0.654
Только результат теста внутри категории	0.335	0.320	0.454	1.68	0.306
Случайный порядок внутри категории	0.330	0.320	0.509	1.69	0.340
BM25 по тексту резюме в специализации	0.199	0.210	0.333	1.15	0.237
BM25 по тексту резюме, вся база («универсальная площадка»)	0.176	0.183	0.318	0.97	0.216



Итог. Механика даёт NDCG@10 = 0.70 против 0.18 у поиска по ключевым словам на «универсальной площадке» и в 4.1 раза больше ожидаемых принятых приглашений в топ-10. При накрутке стека у 30% кандидатов качество BM25 падает на 16%, нашей механики — на 0%.

Опыт работы. У синтетических кандидатов есть история работы: часть джунов без коммерческого опыта, около 20% сменили роль (общий стаж больше релевантного); в потребностях указан требуемый опыт. С учётом релевантного опыта NDCG@10 = 0.700, без него (история скрыта) — 0.613; ожидаемые принятые приглашения в топ-10: 3.95 против 3.32. Оговорка: эталонная релевантность сама понижает кандидатов с нехваткой опыта (так рассуждает работодатель), поэтому прирост показывает согласованность механики с этим правилом, а не независимое подтверждение веса.

v5 против исходной формулы v1 на этой синтетической истине: NDCG@10 0.721 против 0.618, P@10 0.683 против 0.593. Истина генератора считает долю известных технологий вакансии без учёта того, какие из них определяют стек (Go в Go-вакансии весит как Git), — ровно то, что исправляет v5, поэтому здесь v5 не выигрывает. По независимым слепым судьям на реалистичных текстах v5 значимо лучше v1 — bench.md.

v7 (поставка) против v6 и v5. NDCG@10: v7 0.700, v6 0.700, v5 0.721. v6 добавляет «ворота» основного стека (Java-вакансии не нужен C#-разработчик); синтетическая истина важность основного языка не моделирует, поэтому здесь v6 ниже v5, а у слепого асессора на реалистичных текстах — выше (bench2-results.md). v7 поднимает долю ФСП в скоре с 7.5% до 14% и засчитывает достижения ФСП как опыт работы (до года).

Истина генератора понижает кандидатов без стажа (так рассуждает работодатель сегодня), поэтому v7 сознательно расходится с ней — веса выбраны так, чтобы потеря NDCG к v6 не превышала 0.01 (перебор — matching.md).

Что вносит вклад (абляции). Основной прирост дают категоризация по тесту и совместимость условий (без неё NDCG падает сильнее всего). Подоценки навыков из теста дают +0.034 NDCG@10: состав теста строится по заявленному стеку, поэтому заявленные технологии проверяются тестом (при прежнем общем составе подоценки подбору не помогали — см. разложение ниже). ФСП на синтетике NDCG не поднимает (без ФСП — 0.701): в генераторе достижения ФСП коррелируют с уже измеренной тестом способностью, а скрытая истина понижает кандидатов без стажа. В v7 доля ФСП — ≈14% итогового скора (было 7.5%) и ФСП засчитывается как опыт: платформа сознательно поднимает молодых участников соревнований без коммерческого стажа, но не выше вес теста и не в обход «ворот» основного стека.

Состав теста × формула подбора (NDCG@10; в скобках — без подоценок из теста, только заявленный стек):

Состав теста	Формула v1	Формула v5
прежний общий	0.615 (0.617)	0.678 (0.663)
ядро + модули стека	0.618 (0.617)	0.721 (0.692)

Прежняя связка (общий состав + v1) — 0.615, и подоценки из теста ей мешали (-0.002): проверялись навыки не из стека кандидата. Новая (ядро + модули + v5) — 0.721, подоценки дают +0.028.

Чувствительность к весам (изменение NDCG@10 при умножении веса на 0.5 / 1.5):

Вес	×0.5	×1.5
`need_fit`	+0.003	-0.003
`profile`	+0.006	-0.007
`conditions`	-0.014	+0.007
`freshness`	-0.002	-0.001
`ps_test`	+0.000	-0.002
`ps_fsp`	+0.004	-0.007
`coverage_share`	-0.010	-0.009
`claim_base`	-0.005	-0.003
`unclaimed_factor`	-0.006	-0.009
`adjacent_grade_multiplier`	-0.004	-0.048

Базовое NDCG@10 = 0.700. Для 9 из 10 весов изменение в пределах ±50% меняет NDCG меньше чем на 0.04 — формула не «подогнана» под синтетику. Наибольшая чувствительность — к `adjacent_grade_multiplier` (до 0.048): это ожидаемо и подтверждает обоснованность соответствующего правила.

5. Разбор текстов (NLP)

Вакансии. На 7 демонстрационных вакансиях (`ml/fspml/data/vacancies.yaml`) специализация определена верно в 100.0%, грейд — в 100.0%. Blueprint теста под вакансию:

Вакансия	Blueprint (доли навыков в тесте)
fintech-python-middle	SQL и базы данных 23%, Архитектура и проектирование 22%, HTTP, REST и сети 21%, Python 12%, Алгоритмы и структуры данных 11%, Docker 5%
ecom-react-senior	HTML и CSS 36%, JavaScript 30%, React 19%, HTTP, REST и сети 9%, Алгоритмы и структуры данных 6%
telecom-ds-junior	Python 30%, Машинное обучение 28%, Статистика и вероятность 23%, SQL и базы данных 12%, pandas / NumPy 7%
gov-qa-middle	Тест-дизайн и теория тестирования 57%, HTTP, REST и сети 23%, Python 8%, Git 6%, SQL и базы данных 6%
industry-devops-senior	Linux и shell 29%, CI/CD 23%, Docker 15%, HTTP, REST и сети 13%, Архитектура и проектирование 9%, Kubernetes 6%
bank-analyst-middle	Требования и моделирование 50%, SQL и базы данных 27%, HTTP, REST и сети 13%, Продуктовые метрики 6%, Архитектура и проектирование 5%
edtech-android-middle	Мобильные платформы 59%, HTTP, REST и сети 18%, Git 10%, Алгоритмы и структуры данных 6%, Архитектура и проектирование 6%

Резюме. 60 синтетических PDF (разные заголовки секций, форматы дат, опечатки в названиях технологий) → парсер: точность полей name 100.0%, email 100.0%, phone 100.0%, telegram 100.0%, city 100.0%, specialization 100.0%; технологии P/R/F1 = 0.97/1.00/0.99; стаж ±0.5 года — 100.0%. Синтетические PDF чище реальных — следующий шаг: 10–15 реальных обезличенных резюме с ручной разметкой.

6. Ограничения и следующие шаги

- Параметры семейств — экспертные; после запуска — онлайн-перекалибровка (показана выше), мониторинг дрейфа и вывод слабых семейств.
- Живой пилот (15–20 человек): сравнение грейда с экспертной оценкой команды и интервьюеров — протокол в `docs/pilot.md` .
- Эталонная релевантность синтетическая; на реальных данных — обучение весов по откликам на приглашения (learning to rank) с контролем объяснимости.

Бенчмарк подбора: независимая разметка и сильные бейзлайны

> Код: `ml/fspml/bench/` . Данные: `ml/bench/data/` . Результаты на bench2 с v6 и v7 — раздел «Бенчмарк подбора: результаты» (bench2-results.md). Команды: `make bench-build` → `make bench-judge` → `make bench-label` → `make bench-eval` .

Зачем

Прежняя проверка подбора (validation.md §4) сравнивала выдачу с эталоном, построенным из тех же факторов, что и формула ранжирования (специализация, грейд, стек, условия), а в качестве бейзлайна использовала BM25 без фильтров категории. Такой эксперимент показывает, что формула согласована с эталоном, но не доказывает, что она лучше разумных альтернатив. Бенчмарк закрывает оба пробела:

1. **Эталон не выводится из наших признаков.** Релевантность пар размечают LLM-асессоры и эксперты по рубрике, читая полные тексты вакансии и карточки кандидата. Скрытая истина синтетической популяции остаётся третьим, независимым источником.
2. **Сильные бейзлайны при равной информации.** Все системы получают одинаковые тексты и одинаковые жёсткие фильтры категории; различается только упорядочивание.

Методика

Данные

- **Кандидаты** — 1000 синтетических профилей с накруткой стека у части слабых кандидатов. Каждый проходит симулированный адаптивный тест. **Карточка** (`dossier.py`) — то, что видит работодатель: категория, перцентиль теста, статусы навыков по тесту, заявленный стек, резюме, ФСП, условия. Кандидат обезличен (псевдоним `K-xxxx`), скрытая истина в карточку не попадает.
- **Потребности** — 30 синтетических (для них известна истина) и 7 реальных по форме описаний вакансий из `data/vacancies.yaml` (живой текст, разбирается тем же NLP, что и в продукте).

Системы

Система	Что это
`ours`	Механика платформы (`matching/scoring.py`) поверх разбора текста вакансии
`rerank_bge`	Кросс-энкодер BAAI/bge-reranker-v2-m3 поверх top-50 гибрида
`hybrid_rrf`	BM25 + FRIDA, Reciprocal Rank Fusion (k = 60)
`dense_frida`	ai-forever/FRIDA — сильный открытый энкодер для русского языка
`dense_qwen3`	Qwen3-Embedding-0.6B с инструкцией «вакансия → профили»
`bm25`	Okapi BM25 по карточке
`test_only` , `random`	Нижние ориентиры внутри категории

Фильтр категории (специализация и грейд ±1 из разбора вакансии, статус «ищу / открыт») общий для всех систем.

Пул и разметка

Пулинг по схеме TREC: размечается объединение top-10 всех систем для каждой потребности. Кроме того, в пул попадают 4 случайных допустимых кандидата вне пула (оценка упущенного) и 2 **контрольные** пары с кандидатом другой специализации (правильный ответ — 0, проверка внимательности асессора).

Рубрика (`rubric.py`) сформулирована от цели механики, а не от формулы: стоит ли работодателю отправить приглашение, с учётом того, примет ли кандидат приглашение. Шкала 0–3.

LLM-асессор — ансамбль моделей Yandex AI Studio разных семейств (DeepSeek V4.1 Flash, YandexGPT 5.1 Pro, gpt-oss-120b), temperature = 0, слепо к системам. Итоговая метка — округлённое среднее. Все ответы кэшируются: повторный прогон воспроизводим и бесплатен. Данные синтетические; логирование запросов на стороне Яндекса отключено.

Эксперты размечают одну и ту же слепую выборку из 160 пар (стратифицирована по метке Claude, 10 контрольных; набор зафиксирован в `ml/bench/label/sample.json`). Пакет для экспертов — `ml/bench/label/fsp-ekspertnaya-razmetka.zip` : инструкция и автономная страница разметки, работают без интернета. Результаты импортируются командой `python -m fspml.bench import labels-*.json` .

Офлайн-разметка агентами

Облачный LLM используется только во время работы приложения. Офлайн-разметку бенчмарка выполняют агенты, запущенные отдельно (`python -m fspml.bench blind` → `blind-check` → `blind-import`):

- **Claude (claude-opus-5-5)**: 8 независимых агентов, каждый размечает 3–4 пачки по 49–100 пар. Агент видит только файл пачки: рубрику, текст вакансии и карточки. Код механики, данные о системах и вид пары ему недоступны.
- **GPT-5.6 Terra** (агентное приложение, средний уровень рассуждений): та же инструкция `ml/bench/blind/PROMPT_harness.md` ; успел 13 пачек из 25 (861 пара, 19 вакансий целиком). Вторая модель из другого семейства нужна, чтобы проверить, не подыгрывает ли Claude механике.

Каждую вакансию целиком размечает один агент. Строгость агента (средний сдвиг меток) поэтому одинаково действует на все системы внутри вакансии и не искажает сравнение систем, но вносит шум в согласие с экспертами и истиной. Сдвиг по агентам публикуется в отчёте.

Проверка предвзятости. Механику и рубрику писала та же модель, что размечает (Claude). Если асессор подыгрывает механике, его метки на кандидатах, которых нашла *только* наша механика, будут завышены относительно скрытой истины сильнее, чем на кандидатах, найденных только бейзлайнами. Это смещение считается для каждого асессора.

Правило допуска LLM-асессора

LLM-метки используются для сравнения систем, только если согласие LLM ↔ эксперты (взвешенная κ) не ниже согласия экспертов между собой. Дополнительно проверяются: доля нулей на контрольных парах, согласие с истиной на синтетике и совпадение *рейтинга систем* по истине и по LLM (τ Кендалла).

Метрики и статистика

- nDCG@10 *condensed* (неразмеченные кандидаты исключаются из выдачи — система не наказывается за находки вне пула), P@10 (метка ≥ 2), доля размеченных в top-10.
- 95% доверительные интервалы — бутстреп по потребностям; разница с нашей механикой — парный рандомизационный тест.
- Согласие: взвешенная (квадратичная) κ Коэна, α Криппендорфа (интервальная).

Ограничения

- Кандидаты синтетические: тексты резюме шаблонные. Следующий шаг — реалистичные резюме, сгенерированные по скрытому профилю (LLM), чтобы проверить и разбор, и ранжирование на живом языке.
- LLM-асессор может разделять системные искажения с LLM-компонентами будущих версий механики; поэтому контроль по экспертам и по истине обязателен.
- Экспертов в команде мало; согласие экспертов между собой задаёт потолок, к которому можно стремиться, и публикуется вместе с результатами.

Эксперимент: v2 (граф навыков, «ворота») и v3 (LambdaMART)

Анализ ошибок v1 показал, что треть мест в top-10 занимают кандидаты с одним жёстким расхождением (формат, вилка, уровень, обязательные навыки). Проверены две гипотезы:

- **v2** — ручная формула: условия, уровень и обязательные навыки как мультипликативные «ворота», частичный зачёт навыков по графу смежности (`data/skill_graph.yaml` , 323 ребра), ФСП не компенсирует пробелы в обязательном;
- **v3** — дистилляция: LambdaMART (LightGBM) с монотонными ограничениями на понятных признаках, учитель — разметка Claude, прогон строится вне выборки (5 фолдов по вакансиям).

Новые пары, которые подняли v2 и v3 (79), доразмечены вслепую тем же асессором Claude.

Версия	Claude (учитель v3)	GPT-5.6 Terra	Скрытая истина
v1	0.792	0.780	0.687
v2	0.753 (–0.040, p = 0.03)	0.747 (–0.032, p = 0.04)	0.635 (–0.052, p = 0.002)
v3	0.829 (+0.037, p = 0.006)	0.762 (–0.018, p = 0.24)	0.687 (0.000)

Абляция v2 (nDCG@10, Claude / GPT / истина): без «ворот» — 0.455 / 0.502 / 0.327; без графа — 0.730 / 0.757 / 0.641. «Ворота» работают, проигрыш v2 — в базовой части формулы: условия убраны из аддитивной части, и пропала градация внутри вилки. Граф навыков на синтетике эффекта не дал: в синтетической популяции стек кандидата не содержит реалистичных корреляций технологий, а скрытая истина вовсе не засчитывает смежные технологии, — проверить граф можно только на реалистичных резюме и экспертной разметке.

Вывод. Ручная доработка формулы по анализу ошибок ухудшила результат у всех судей — это и есть причина, по которой каждое изменение проверяется бенчмарком. Обученная v3 улучшает результат только по меткам своего учителя; по независимым судьям она на уровне v1. Прирост по учителю не засчитывается. В продукте оставалась v1 (до v5 — см. ниже); v2 и v3 доступны как экспериментальные (`rank(..., version=2)` , `matching/ltr.py`).

Реалистичный бенчмарк (bench2)

700 кандидатов и 30 вакансий; стеки — из архетипов (`data/stack_archetypes.yaml`), тексты резюме и вакансий написаны агентами по скрытым профилям; «заявленный стек» в карточке получен разбором текста резюме нашим NLP. Разбор живых текстов: технологии в резюме F1 = 0.983, в вакансиях F1 = 0.956, специализация и грейд вакансии — 30/30 (до исправления словоформ и «ведущий» → senior: вакансии F1 = 0.951, грейд 28/30). Разметка: слепые агенты Claude, 1278 пар.

Система	Claude, nDCG@10	Скрытая истина
v1	0.714	0.768
v2	0.696 (p = 0.33)	0.719
v3	0.731 (p = 0.55)	0.668
v1 ⊕ кросс-энкодер (RRF, вес v1 ×2)	**0.747** (+0.033, p = 0.006)	0.756
кросс-энкодер bge-reranker-v2-m3	0.649 (p = 0.09)	0.519
гибрид BM25 + FRIDA	0.657 (p = 0.13)	0.512
BM25	0.585 (p = 0.02)	0.439

Выводы. На живых текстах нейросетевые модели читают резюме гораздо лучше, и по независимому судье отрыв v1 от них сокращается до 0.06–0.08 и перестаёт быть значимым. Скрытая истина по-прежнему показывает отрыв 0.25, но проверка смещения подтверждает, что она подыгрывает формуле: на находках v1 судья ставит в среднем на 0.73 балла ниже истины, на находках бейзлайнов — на 0.23 выше. Поэтому главным эталоном считаются независимые судьи и эксперты. Лучший результат на тот момент — **гибрид**: структурная механика (категория, условия, подтверждённые навыки) плюс семантическое понимание текста кросс-энкодером (v5 и гибрид на ней — в следующем разделе). Вес смешивания выбран из двух вариантов на тех же данных — нужна проверка вторым судьёй и экспертами.

Эксперимент: v5 (специфичность требований и «ворота»)

Разбор ошибок v1 на bench2. В топ-10 v1 — 148 из 300 мест с оценкой асессора 0–1. Около 60% из них — **несовпадение основной технологии**: на «Senior Go» — Java-, Python- и Kotlin-лиды, на PHP/Laravel — junior на Java, C#, Go, Node.js, на Vue — React- и Angular-разработчики, на iOS — Android, на Java+Selenium+Appium — автоматизаторы на Python и ручные тестировщики, на CV — табличный DS, на продуктового аналитика — системные. Механизм: в покрытии все требования весили одинаково, поэтому Java-разработчик с Docker, Git, PostgreSQL и Kafka закрывал PHP-вакансию почти так же, как PHP-разработчик (покрытие 0.41 против 0.43), а сила профиля и условия решали остальное. Вторая группа — несовместимые условия (формат, ожидания сильно выше вилки), которые аддитивная формула не отсекает.

Ошибка разбора вакансий. Попутно найдено, что однострочное описание («Обязательно: …. Будет плюсом: …») целиком уходило в «будет плюсом», а короткие пункты списка с маркером («- Знание Redis обязательно») считались заголовками и терялись. Все синтетические вакансии bench1 однострочные, поэтому «ворота» обязательного v2 на bench1 фактически не срабатывали — вывод о v2 на bench1 этой ошибкой частично искажён. После исправления доля верно размеченных «обязательно / плюс» на однострочных вакансиях выросла с 36% до 62% (остаток — рассинхронизация сохранённых текстов с перегенерированной истиной; разбор следует тексту), на живых текстах bench2 не изменилась (96%).

Гипотеза и варианты (заданы до прогона, коэффициенты не подбирались):

- **специфичность** требования внутри пула категории — √(–ln доли кандидатов, заявивших технологию), как IDF в поиске: Git и Docker есть у всех и не отличают кандидатов, Go и Vue определяют стек вакансии;
- «**ворота» v2** (обязательное, формат, вилка) поверх аддитивной формулы v1. В отличие от v2 условия остаются и в аддитивной части (градация внутри вилки сохраняется), граф смежности не используется (в v2 он ухудшал результат), покрытие обязательного взвешено специфичностью.

Результаты (nDCG@10 condensed; Δ и p — к v1 с тем же разбором вакансий):

Система	bench2, Claude	bench2, истина	bench1, GPT-5.6 Terra	bench1, Claude	bench1, истина
v1 (прежний разбор вакансий)	0.704	0.768	0.778	0.791	0.686
v1	0.704	0.767	0.789	0.801	0.690
v5	**0.792** (+0.088, p < 0.001)	0.729 (–0.038, p = 0.01)	**0.803** (+0.013, p = 0.07)	**0.843** (+0.042, p = 0.002)	0.700 (+0.010, p = 0.62)
v5 ⊕ кросс-энкодер	**0.813** (+0.108, p < 0.001)	0.750 (–0.017, p = 0.26)	0.796 (+0.006, p = 0.79)	0.813 (+0.012, p = 0.49)	0.685 (–0.005, p = 0.76)
v1 ⊕ кросс-энкодер (v4)	0.736 (+0.032, p = 0.008)	0.767	—	—	—

Абляции v5 (Δ nDCG@10 к v1; bench2 Claude / bench1 GPT / bench1 Claude):

Вариант	bench2, Claude	bench1, GPT	bench1, Claude
только специфичность	+0.024	0.000	–0.002
только «ворота» условий (формат, вилка)	+0.002	+0.016	+0.042
только «ворота» обязательного	+0.030	–0.001	+0.005
все «ворота», без специфичности	+0.043	+0.010	+0.048
специфичность + «ворота» обязательного	+0.064	–0.008	+0.001
v5 целиком	**+0.088**	**+0.013**	**+0.042**

На реалистичных стеках (bench2) решает связка «специфичность + ворота обязательного»: по отдельности каждая даёт +0.02–0.03, вместе +0.064. На синтетике bench1 технологии вакансии выбраны независимо друг от друга, «основного языка» там нет, и выигрыш дают «ворота» условий. Полная v5 — единственный вариант с приростом у всех трёх LLM-судей.

Форма веса специфичности. ln: +0.106 / +0.019 / +0.037; ln с потолком 2: +0.103 / +0.013 / +0.031; 0.5 + 0.5·ln: +0.081 / +0.014 / +0.040; √: +0.088 / +0.013 / +0.042 (bench2 Claude / bench1 GPT / bench1 Claude). Выигрыш от формы не зависит; выбран √ — мягкий вариант, который меньше других расходится со скрытой истиной (bench2: –0.038 против –0.067 у ln) и меньше перевешивает редкие второстепенные метки вроде «Интеграции».

Почему скрытая истина bench2 против. Генератор истины считает долю известных кандидату технологий вакансии без учёта того, какие из них определяют стек, и сильно штрафует соседний грейд. Поэтому на вакансии «Senior Golang-разработчик» он ставит «3» Java-тимлиду без Go («Java тимлид без Go, ожидания выше вилки» — 0 у асессора), а на «Senior QA Automation (Java)» — «0» автоматизаторам Java+Selenium+Appium уровнем ниже (2 у асессора) и «2» ручному тестировщику без автоматизации (0 у асессора). Ровно эти перестановки делает v5. Это та же предвзятость истины к исходной формуле, что зафиксирована выше проверкой смещения.

Ограничения. Разбор ошибок делался по меткам Claude на bench2, поэтому прирост на bench2 — внутривыборочный; варианты при этом зафиксированы до прогона, а bench1 (другая популяция, второй судья из другого семейства) подтверждает направление эффекта. У GPT-5.6 Terra размечено 19 вакансий, и прирост +0.013 там не значим (p = 0.07). В топ-10 новых систем 3–9% пар не размечены — они добавлены в пул для слепой доразметки (`added: true`; bench2 — 5 пар, bench1 — 15 пар): `python -m fspml.bench blind` → разметка агентами → `blind-import`. Следующий шаг — экспертная разметка тех же пар.

Вывод. По итогам эксперимента в продукт вошла **v5**; гибрид с кросс-энкодером (`FSP_RANKER=hybrid`) даёт ещё +0.02 на живых текстах, но требует модели и ничего не добавляет на шаблонных; v1 доступна как `version=1` . Сейчас в поставке **v7** — v5 с «воротами» основного стека и весомым ФСП (matching.md, §4.5).

Бенчмарк подбора: результаты слепой разметки

> Сгенерировано `make bench-eval`. Методика — bench.md.

Потребностей: 30 (синтетических 30, реальных описаний вакансий 0); кандидатов: 700; пар в пуле: 1327.

Сводка: все судьи на одних и тех же 30 вакансиях

nDCG@10. Числа сравниваются только внутри столбца: у судей разная строгость шкалы.

Система	claude-opus-5-5	скрытая истина
Наша механика v1	0.704	0.768
Наша механика v2 (граф навыков, «ворота»)	0.686	0.719
Наша механика v3 (LambdaMART, дистилляция)	0.725	0.668
Наша механика v4 (гибрид v1 + кросс-энкодер)	0.734	0.767
Наша механика v5 (специфичность требований + «ворота»)	0.789	0.729
Наша механика v5 + кросс-энкодер (гибрид)	0.807	0.750
Наша механика v6 (v5 + «ворота» основной технологии)	0.812	0.665
Наша механика v7 (v6 + весомый ФСП, ФСП как опыт)	0.817	0.668
Кросс-энкодер bge-reranker-v2-m3 (поверх гибрида)	0.640	0.519
Гибрид BM25 + FRIDA (RRF)	0.649	0.512
Эмбеддинги FRIDA	0.626	0.504
Эмбеддинги Qwen3-Embedding-0.6B	0.649	0.449
BM25 по карточке	0.579	0.439
Только результат теста	0.210	0.187
Случайный порядок	0.275	0.243

Результаты по каждому судье

Скрытая истина (30 синтетических вакансий)

Система	nDCG@10 [95% ДИ]	P@10	Размечено в top-10	Δ к v1	p
Наша механика v1	0.768 [0.74–0.80]	0.700	1.00	—	—
Наша механика v2 (граф навыков, «ворота»)	0.719 [0.67–0.76]	0.643	1.00	-0.049	0.0042
Наша механика v3 (LambdaMART, дистилляция)	0.668 [0.62–0.72]	0.607	1.00	-0.100	0.0005
Наша механика v4 (гибрид v1 + кросс-энкодер)	0.767 [0.73–0.80]	0.687	1.00	-0.002	0.8916
Наша механика v5 (специфичность требований + «ворота»)	0.729 [0.68–0.77]	0.660	1.00	-0.039	0.0102
Наша механика v5 + кросс-энкодер (гибрид)	0.750 [0.71–0.79]	0.663	1.00	-0.018	0.2165
Наша механика v6 (v5 + «ворота» основной технологии)	0.665 [0.60–0.72]	0.573	1.00	-0.103	0.0001
Наша механика v7 (v6 + весомый ФСП, ФСП как опыт)	0.668 [0.60–0.73]	0.583	1.00	-0.100	0.0001
Кросс-энкодер bge-reranker-v2-m3 (поверх гибрида)	0.519 [0.46–0.58]	0.470	1.00	-0.249	0.0000
Гибрид BM25 + FRIDA (RRF)	0.512 [0.45–0.57]	0.417	1.00	-0.256	0.0000
Эмбеддинги FRIDA	0.504 [0.44–0.57]	0.440	1.00	-0.265	0.0000
Эмбеддинги Qwen3-Embedding-0.6B	0.449 [0.39–0.50]	0.383	1.00	-0.319	0.0000
BM25 по карточке	0.439 [0.38–0.50]	0.353	1.00	-0.330	0.0000
Только результат теста	0.187 [0.11–0.28]	0.163	1.00	-0.581	0.0000
Случайный порядок	0.243 [0.19–0.31]	0.247	1.00	-0.526	0.0000

Судья claude-opus-5-5

Размечено пар: 1296; вакансий целиком: 30 (из них живых: 0).

Система	nDCG@10 [95% ДИ]	P@10	Размечено в top-10	Δ к v1	p
Наша механика v1	0.704 [0.65–0.76]	0.507	1.00	—	—
Наша механика v2 (граф навыков, «ворота»)	0.686 [0.63–0.73]	0.490	1.00	-0.019	0.3114
Наша механика v3 (LambdaMART, дистилляция)	0.725 [0.68–0.77]	0.557	0.92	0.021	0.4461
Наша механика v4 (гибрид v1 + кросс-энкодер)	0.734 [0.68–0.78]	0.527	0.98	0.030	0.0078

Система	nDCG@10 [95% ДИ]	P@10	Размечено в top-10	Δ к v1	p
Наша механика v5 (специфичность требований + «ворота»)	0.789 [0.75–0.83]	0.603	1.00	0.085	0.0000
Наша механика v5 + кросс-энкодер (гибрид)	0.807 [0.77–0.84]	0.590	0.99	0.103	0.0000
Наша механика v6 (v5 + «ворота» основной технологии)	0.812 [0.77–0.86]	0.600	1.00	0.108	0.0003
Наша механика v7 (v6 + весомый ФСП, ФСП как опыт)	0.817 [0.77–0.86]	0.597	1.00	0.113	0.0002
Кросс-энкодер bge-reranker-v2-m3 (поверх гибрида)	0.640 [0.58–0.70]	0.430	1.00	-0.064	0.0941
Гибрид BM25 + FRIDA (RRF)	0.649 [0.60–0.70]	0.410	1.00	-0.055	0.1375
Эмбеддинги FRIDA	0.626 [0.57–0.69]	0.410	1.00	-0.078	0.0531
Эмбеддинги Qwen3-Embedding-0.6B	0.649 [0.58–0.71]	0.437	1.00	-0.055	0.2689
BM25 по карточке	0.579 [0.51–0.65]	0.370	1.00	-0.125	0.0179
Только результат теста	0.210 [0.14–0.29]	0.113	1.00	-0.494	0.0000
Случайный порядок	0.275 [0.22–0.34]	0.177	1.00	-0.429	0.0000

Согласие асессоров

Сравнение	n	к взв.	точное	±1	Спирмен
claude-opus-5-5 ↔ скрытая истина	1296	0.55	0.46	0.91	0.53

Проверка предвзятости судьи (средний сдвиг «оценка судьи – скрытая истина»). Если судья подыгрывает нашей механике, сдвиг на её находках будет больше, чем на находках бейзлайнов. Отрицательный сдвиг на находках механики у всех судей означает обратное: скрытая истина, построенная из тех же факторов, что и формула, сама немного завышает находки механики.

Асессор	Только наша механика: n / смещение	Только бейзлайны: n / смещение
claude-opus-5-5	103 / -0.73	446 / 0.23

- α Криппендорфа: судьи-LLM: **—**
- Контрольные пары, оценка 0 — claude-opus-5-5: **1.00**
- τ Кендалла: рейтинг систем по истине и по claude-opus-5-5: **0.60**
- Доля релевантных среди случайных пар вне пула: **0.05**

Витрина: один кандидат — два порядка выдачи

> Сгенерировано `python -m fspml.validation.showcase`. Популяция и потребности — синтетическая валидация (validation.md); формулы — matching.md.

Вакансия: Data Science · Junior требуемые технологии (важность): NLP (1.00), PyTorch (1.00), Scala (1.00), pandas (1.00), Airflow (0.45), Machine learning (0.45), Рекомендательные системы (0.45); опыт — от 1 г (указан в вакансии).

Главное. Кандидат `c0724` — участник ФСП (участник, всероссийские, 2024 (+3)), релевантный опыт 0.8 г. У кандидата `c0835` тест выше (θ -0.32 против -0.35), опыт 1.5 г, ФСП: участник, всероссийские, 2024. В v5 `c0724` на 3-м месте, `c0835` — на 2-м; в v7 `c0724` поднимается на 1-е, `c0835` — 2-е.

Почему: разложение скора

	v5: c0724	v5: c0835	v7: c0724	v7: c0835
Место в выдаче	3	2	1	2
Итоговый скор	0.368	0.384	0.389	0.362
Вклад силы профиля	0.087	0.095	0.095	0.086
— тест (0..1)	0.81	0.84	0.81	0.84
— ФСП (0..1)	0.22	0.07	0.22	0.07
Вклад соответствия навыкам	0.129	0.111	0.125	0.097
Множитель опыта	×0.90	×1.01	×1.00	×1.01

Доля ФСП в итоговом скоре: v5 — 0.30 × 0.25 = 7.5%, v7 — 0.35 × 0.40 = 14%. Сила ФСП `c0724`: 0.22.

Что видит работодатель в карточке `c0724` (v7)

- ✅ Подтверждено тестом: Airflow, Machine learning, NLP, pandas
- ⚪ Заявлено, но не проверено: Scala
- ✅ Тест: 60-й перцентиль категории
- ⚪ Тест без проктора: результат не подтверждён наблюдением
- ⚪ Релевантный опыт: меньше года; требуется от 1 года
- ✅ Достижения ФСП засчитаны как практический опыт (≈ 2 мес.)
- ✅ ФСП: участник — всероссийские соревнования, программирование систем информационной безопасности, 2024 (+3)
- ⚪ Вилка частично пересекается с ожиданиями
- ✅ Активен 7 дн. назад
- ⚠️ Не подтверждено: PyTorch

Топ-10: v5 и v7

#	v5	θ	ФСП	опыт	v7	θ	ФСП	опыт
1	c0896	-0.38	ID не привязан	0.7 г	**c0724**	-0.35	участник, всероссийские, 2024 (+3)	0.8 г
2	**c0835**	-0.32	участник, всероссийские, 2024	1.5 г	**c0835**	-0.32	участник, всероссийские, 2024	1.5 г
3	**c0724**	-0.35	участник, всероссийские, 2024 (+3)	0.8 г	c0090	-0.44	финалист, региональные, 2026 (+1)	0.5 г
4	c0216	-0.80	ID не привязан	1.0 г	c0545	-0.44	ID не привязан	1.4 г
5	c0096	-0.10	участник, региональные, 2023	0.9 г	c0659	-0.88	ID не привязан	1.2 г
6	c0545	-0.44	ID не привязан	1.4 г	c0073	+0.50	ID не привязан	3.6 г
7	c0090	-0.44	финалист, региональные, 2026 (+1)	0.5 г	c0552	+0.44	призёр, всероссийские, 2025	3.2 г
8	c0378	-0.46	участник, региональные, 2025 (+2)	1.8 г	c0212	+1.04	ID не привязан	3.2 г
9	c0035	-0.08	участник, всероссийские, 2022 (+3)	1.4 г	c0988	+0.58	участник, всероссийские, 2023 (+2)	3.1 г
10	c0789	+0.43	ID не привязан	1.5 г	c0953	+0.43	ID не привязан	3.6 г

Где категория выше теста: внутри одной категории порядок по-прежнему начинается с теста (вес теста в силе профиля 0.45 ≥ веса ФСП 0.40), а достижения ФСП решают, когда тесты близки и у участника соревнований нет коммерческого стажа. «Ворота» основного стека ФСП не обходит.

bench3: живые вакансии, v5 против v7

> Сгенерировано `python -m fspml.bench.live`. 10 реальных открытых вакансий Junior/Middle с Хабр Карьеры (тексты — `ml/bench3/vacancies.json`, без названий компаний) × 20 обезличенных собирательных профилей (`ml/bench3/candidates.yaml`: стек, стаж и достижения — типовые для открытых резюме и итоговых таблиц ФСП; реальных людей нет). Иллюстрация на малой выборке, не статистика.

Разбор живых текстов нашим NLP. Специализация верна в 9 из 10 (эталон — специализация на площадке); точность грейда — в разделе ниже, на 50 вакансиях. Ошибки специализации:

- «Аналитик по данным AI/ML»: разбор — analyst, на площадке — data_science

В продукте разбор показывается работодателю с объяснением и исправляется вручную; в выдаче ниже специализация и грейд — с площадки.

Грейд по тексту вакансии: dev и test

50 реальных вакансий Хабр Карьеры; эталон — квалификация, которую работодатель указал на площадке. Грейд — взвешенное голосование сигналов: заголовок (решает сам), требуемый стаж, уровень в тексте без упоминаний коллег, лексика уровня, вилка относительно медиан специализации (`grade_salary` в каталоге). Уверенность: high — заголовок или согласие ≥2 сигналов, medium — один сигнал, low — конфликт или сигналов нет (тогда грейд не предлагается и работодатель выбирает его вручную). Правила настраивались только на dev.

Часть	Вакансий	Грейд верен	Верен при high	Доля low	Без грейда в заголовке: верно
dev	31	25 из 31 (81%)	19 из 20	6%	15 из 20
test	19	14 из 19 (74%)	12 из 13	16%	8 из 13

Ошибки dev:

- «Java-разработчик» (h165768): разбор — Senior [medium], на площадке — Middle; сигналы: требуемый опыт от 3 лет
- «Backend-разработчик (Java Spring / Node.js)» (h169081): разбор — не определён [low], на площадке — Junior; сигналы: уровень в тексте не указан — выберите вручную
- «QA Automation Engineer (Java)» (h168829): разбор — Middle [medium], на площадке — Junior; сигналы: требуемый опыт от 1,6 года; против: лексика Junior: «базовые знания»
- «Технолог по ИТ-процессам» (h167830): разбор — не определён [low], на площадке — Middle; сигналы: уровень в тексте не указан — выберите вручную
- «Старший разработчик серверной части (C#) [Telco Platform, Компания Веб Сервисы]» (h168153): разбор — Senior [high], на площадке — Middle; сигналы: в заголовке указан уровень «Senior»
- «Data Engineer» (h168887): разбор — Senior [medium], на площадке — Lead; сигналы: требуемый опыт от 5 лет

Ошибки test:

- «Инженер-тестировщик (QA WebUI)» (h168205): разбор — Middle [medium], на площадке — Junior; сигналы: требуемый опыт от 1 года; против: лексика Junior: «базовые знания»
- «QA инженер (java/kotlin)» (h128807): разбор — Middle [medium], на площадке — Junior; сигналы: лексика Middle: «самостоятельно»
- «ML Разработчик» (h168375): разбор — не определён [low], на площадке — Senior; сигналы: уровень в тексте не указан — выберите вручную
- «Android Developer» (h167985): разбор — Middle [high], на площадке — Senior; сигналы: требуемый опыт от 1,5 года; лексика Middle: «самостоятельно», ««опыт …» в 5 пунктах требований»
- «Инженер-разработчик по надёжности и эксплуатации (SRE-инженер)» (h169265): разбор — не определён [low], на площадке — Lead; сигналы: уровень в тексте не указан — выберите вручную

Выдача по вакансиям

Жирным — участники ФСП с релевантным стажем меньше года. ↑/↓ — сдвиг позиции в v7 относительно v5.

`QA engineer junior (QA · Junior, вилка не указана)`

Источник

v5: q1 → q5 → q3 → **q2** → q4

v7: q1 → q5 → **q2** ↑1 → q3 ↓1 → q4

> q2 в v7 — 3-е место: Нет коммерческого опыта; требуется от 1 года; Достижения ФСП засчитаны как практический опыт (≈ 3 мес.); ФСП: призёр — региональные соревнования, продуктовое программирование, 2025 (+1)

`Python Developer (Backend · Junior, от 75 000 ₽)`

Источник

v5: b2 → **b1** → **b3** → b4 → **b5** → b7 → b6 → b8

v7: **b1** ↑1 → b2 ↓1 → **b3** → b7 ↑2 → b6 ↑2 → **b5** ↓1 → b4 ↓3 → b8

> b1 в v7 — 1-е место: Нет коммерческого опыта; для Junior обычно от полугода; Достижения ФСП засчитаны как практический опыт (≈ 5 мес.); ФСП: призёр — всероссийские соревнования, продуктовое программирование, 2025

Java-разработчик (Backend · Middle, вилка не указана)

Источник

v5: b8 → b6 → b4 → b7 → b2 → **b5** → **b1** → **b3**

v7: b8 → **b5** ↑4 → b4 → b6 ↓2 → b7 ↓1 → b2 ↓1 → **b1** → **b3**

> b5 в v7 — 2-е место: Нет коммерческого опыта; требуется от 3 лет; Достижения ФСП засчитаны как практический опыт (≈ 7 мес.); ФСП: победитель — всероссийские соревнования, продуктовое программирование, 2025

Java-разработчик (Backend · Junior, вилка не указана)

Источник

v5: b4 → b8 → b2 → **b5** → b7 → b6 → **b1** → **b3**

v7: b4 → **b5** ↑2 → b8 ↓1 → b2 ↓1 → **b1** ↑2 → **b3** ↑2 → b7 ↓2 → b6 ↓2

> b5 в v7 — 2-е место: Нет коммерческого опыта; требуется от полугода; Достижения ФСП засчитаны как практический опыт (≈ 6 мес.); ФСП: победитель — всероссийские соревнования, продуктовое программирование, 2025

Аналитик по данным AI/ML (Data Science · Middle, вилка не указана)

Источник

v5: d1 → **d2**

v7: d1 → **d2**

> d2 в v7 — 2-е место: Релевантный опыт: меньше года; требуется от 1 года; Достижения ФСП засчитаны как практический опыт (≈ 5 мес.); ФСП: победитель — всероссийские соревнования, алгоритмическое программирование, 2025

Инженер по автоматизации тестирования (Java/Selenium) / QA Automation (QA · Middle, вилка не указана)

Источник

v5: q4 → q5 → q1 → q3 → **q2**

v7: q5 ↑1 → q4 ↓1 → q1 → q3 → **q2**

> q2 в v7 — 5-е место: Нет коммерческого опыта; требуется от 1,5 года; Достижения ФСП засчитаны как практический опыт (≈ 3 мес.); ФСП: призёр — региональные соревнования, продуктовое программирование, 2025 (+1)

Системный аналитик (Analyst · Junior, 100 000–130 000 ₽)

Источник

v5: a1 → **a2**

v7: a1 → **a2**

> a2 в v7 — 2-е место: Нет коммерческого опыта; для Junior обычно от полугода; Достижения ФСП засчитаны как практический опыт (≈ 2 мес.); ФСП: призёр — региональные соревнования, продуктовое программирование, 2025

AQA Тестировщик (Python) (QA · Junior, 75 000–90 000 ₽)

Источник

v5: q3 → **q2** → q1 → q5 → q4

v7: q3 → **q2** → q1 → q5 → q4

> q2 в v7 — 2-е место: Нет коммерческого опыта; для Junior обычно от полугода; Достижения ФСП засчитаны как практический опыт (≈ 3 мес.); ФСП: призёр — региональные соревнования, продуктовое программирование, 2025 (+1)

Python backend (Backend · Middle, от 230 000 ₽)

Источник

v5: b6 → b7 → b2 → b8 → **b1** → b4 → **b5** → **b3**

v7: b6 → b7 → b2 → **b1** ↑1 → **b3** ↑3 → b8 ↓2 → **b5** → b4 ↓2

> b1 в v7 — 4-е место: Нет коммерческого опыта; требуется от 2 лет; Достижения ФСП засчитаны как практический опыт (≈ 5 мес.); ФСП: призёр — всероссийские соревнования, продуктовое программирование, 2025

DevOps инженер (DevOps · Middle, вилка не указана)

Источник

v5: o1 → **o2** → o3

v7: o1 → **o2** → o3

> o2 в v7 — 2-е место: Релевантный опыт: меньше года; для Middle обычно от 2 лет; Достижения ФСП засчитаны как практический опыт (≈ 5 мес.); ФСП: призёр — всероссийские соревнования, программирование систем информационной безопасности, 2025

Итог

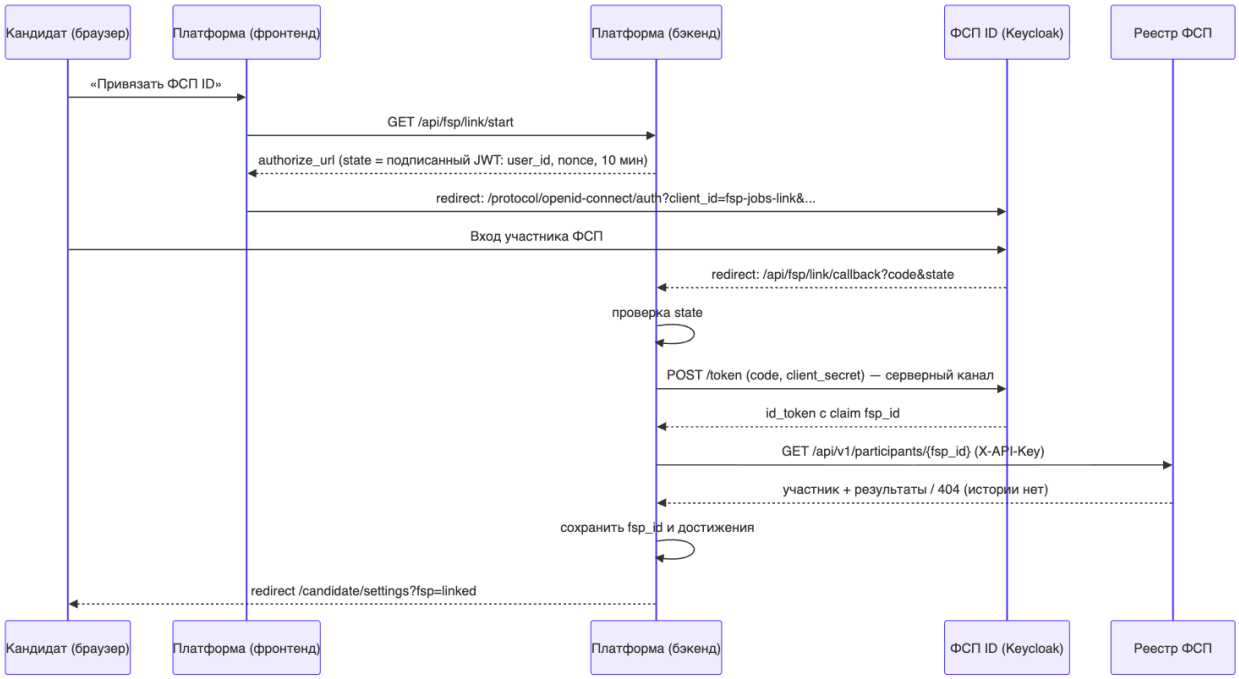
Участники ФСП со стажем меньше года появились в выдаче 18 раз; в v7 они поднялись относительно v5 в 8 случаях из 18. Кандидаты с опытом и сильным тестом остаются наверху, если стек совпадает: v7 не меняет «ворота» основного стека и вес теста (тест ≥ ФСП в силе профиля).

Схема интеграции с ФСП ID и реестром ФСП

ТЗ: API и структура данных ФСП не предоставляются — команда сама решает, какие сведения получать и как показывать; обязательна корректная обработка отсутствия истории. Ниже — предлагаемая схема. В демо она работает на имитациях: realm `fsp-id` в Keycloak (ФСП ID) и сервис `fsp-registry-mock` (реестр).

1. Привязка профиля к ФСП ID

ФСП ID будет работать на Keycloak, поэтому привязка — стандартный OAuth 2.0 / OpenID Connect *authorization code flow*. Платформа выступает OIDC-клиентом `fsp-jobs-link` (конфиденциальный, с секретом).



Переход на реальный ФСП ID — смена `KEYCLOAK_PUBLIC_URL` / `FSP_ID_REALM` / `FSP_ID_CLIENT_ID` / `FSP_ID_CLIENT_SECRET` и маппинга claim `fsp_id`. Один ФСП ID привязывается только к одному профилю (409 при попытке повторной привязки). Отвязка удаляет идентификатор и достижения.

2. Предлагаемые данные реестра

Сущность	Поля	Зачем платформе
Участник	`fsp_id`, ФИО, регион, спортивный разряд/звание, `updated_at`	Сверка личности, разряд — дополнительный сигнал
Соревнование	`id`, название, дисциплина (`algorithmic`, `product`, `security`, `robotics`, `uav`), уровень (`international`, `national`, `regional`, `local`), даты, город, организатор	Вес достижения и релевантность специализации
Результат	`id`, `event_id`, статус (`winner`, `prize`, `finalist`, `participant`), место, дата, роль в команде, команда	Сила достижения, роль (backend/frontend/капитан…) для работодателя

Формат ответа `GET /api/v1/participants/{fsp_id}`:

```
{
  "fsp_id": "FSP-000123",
  "full_name": "Иванова Мария",
  "region": "Москва",
  "sport_rank": "КМС",
  "updated_at": "2026-09-30T00:00:00",
  "achievements": [
    {"id": "FSP-000123-R1", "event_id": "EV-0007",
     "event": "Всероссийские соревнования по продуктовому программированию 2025",
     "discipline": "product", "level": "national", "result": "prize", "place": 2,
     "date": "2025-11-20", "role": "backend", "team": "Команда «Сигма»"}
  ]
}
```

- `404` — участника нет в реестре: **история ФСП отсутствует**, профиль полноценный.
- Синхронизация: при привязке, по кнопке «Обновить» и фоновой задачей (раз в `FSP_SYNC_INTERVAL_HOURS`, по умолчанию 6 ч); для масштаба — дельта `GET /api/v1/participants?changed_since=…`.
- Аутентификация сервис-сервис — API-ключ (в продакшене — mTLS или client credentials того же Keycloak).
- Справочник соревнований: `GET /api/v1/events` (`?upcoming=true` — предстоящие, для плана роста кандидата).

3. Как достижения используются и показываются

- **Ранжирование:** компонент `fsp` силы профиля (40% силы профиля, 14% итогового скора в формуле v7) — noisy-OR по достижениям с учётом уровня, результата, релевантности дисциплины специализации и давности (полураспад 2 года). Матрица релевантности дисциплин — `catalog.yaml` (например, продуктовое программирование → Backend/Frontend/Mobile 1.0; ИБ → DevOps 1.0).
- **Карточка кандидата:** «ФСП: призёр — всероссийские соревнования, продуктовое программирование, 2025 (+1)», список достижений с ролью в команде, фильтр «есть подтверждённые достижения ФСП», число кандидатов с ФСП в каждой категории.
- **Отсутствие истории** — нейтрально: «ФСП ID не привязан» или «ФСП ID привязан, достижений пока нет»; без штрафа к профилю.
- **Приватность:** кандидат может скрыть достижения; скрытые не участвуют и в ранжировании.

4. Трудоустройство участников: что платформа делает для ФСП

Задача платформы — пристроить участников ФСП. Кроме веса достижений в подборе (v7) и ФСП-приора в тесте, для этого есть четыре механики.

Механика	Для кого	Как работает
Вакансии «открыты для участников ФСП»	работодатель → кандидат	Флаг `fsp_friendly` у вакансии и подборки: достижения ФСП засчитываются как опыт до требуемого стажа (обычно — до 1 года, `FSP_YEARS_MAX`). В ленте кандидата — бейдж и фильтр, у компании с такой вакансией — бейдж «Партнёр ФСП».
Подписка на новых участников ФСП	работодатель	Флаг `notify_fsp` у подборки. Когда у участника ФСП подтверждается грейд или обновляются достижения, он пересчитывается в подборках своей специализации. Если попал в топ-10, работодатель получает письмо-дайджест (не чаще раза в сутки на подборку) и вебхук `candidate.fsp_match` — без контактов, только анонимная карточка и причина.
Что доказывает достижение	работодатель	В карточке и PDF у достижения — формат дисциплины и что оно подтверждает (`catalog.yaml`, блок `fsp.disciplines`), команда, роль, место и ссылка на проект команды (вводит кандидат, сохраняется при обновлении из реестра). После теста у кандидата — «план роста»: навыки, которые стоит подтянуть, и ближайшие соревнования ФСП по специализации.
Воронка трудоустройства	федерация (роль `fsp_admin`)	`GET /api/fsp/funnel`, страница «Трудоустройство участников ФСП». Этапы: регистрация → опрос → подтверждённый грейд → приглашение или отклик → договорились о контакте → вышли на работу (работодатель отмечает по принятому приглашению). Сравнение с кандидатами без ФСП, разрезы по уровню и дисциплине соревнования, специализации и региону. Только агрегаты; группы меньше 5 человек не раскрываются; кандидаты, скрывшие ФСП, не учитываются.

5. Вопросы к ФСП для реальной интеграции

Схема реестра выше — наше предложение. Для подключения к настоящему реестру и ФСП ID нужно согласовать:

1. Какие сущности есть в реестре: соревнования, результаты, команды, роли в команде; есть ли результаты отборочных этапов.
2. Классификация соревнований по уровню и официальный вес в рейтинге ФСП, который можно переиспользовать вместо наших коэффициентов.
3. Матрица релевантности дисциплин специализациям (сейчас — `catalog.yaml`).
4. С какого года данные полны и достоверны; как часто обновляется реестр и есть ли выборка «изменённые после даты».
5. Claim с идентификатором участника в токене ФСП ID, его формат и стабильность; отдельный клиент для внешних платформ.
6. Какие согласия участник даёт при привязке и кто оператор ПДн на каждом этапе. Сейчас согласие на передачу достижений в платформу получается в момент привязки.

Описание API

> Сгенерировано из OpenAPI-схемы (`docs/tools/gen_docs.py`). Интерактивная документация — **Swagger UI** `/api/docs` , ReDoc — `/api/redoc` , схема — `/api/openapi.json` и openapi.json.

Общие соглашения

- **Аутентификация:** `Authorization: Bearer <access_token>` — токен OIDC из Keycloak (realm `fsp-jobs` , клиент `fsp-jobs-web` , PKCE). Бэкенд проверяет подпись по JWKS, срок, issuer и audience `fsp-jobs-api` . Публичные методы: справочники, конфигурация авторизации, демо «тест из вакансии».
- **Роли:** кандидат / работодатель — выбираются при первом входе (`POST /api/me/role`). Чужие объекты возвращают `404` (не раскрываем факт существования), неверная роль — `403` .
- **Ошибки:** единый формат `{"error": {"code": "...", "message": "...", "details": ...}}` .

HTTP	Пример code	Когда
400	`consent_required` , `specialization_required` , `pdf_unreadable`	Некорректный запрос
401	`unauthorized` , `invalid_token`	Нет или недействителен токен
403	`wrong_role` , `email_not_verified`	Недостаточно прав
404	`not_found`	Объект не найден или чужой
409	`attempt_not_allowed` , `already_invited` , `survey_required` , `fsp_id_taken`	Конфликт состояния (кулдаун, повтор, нет предусловия)
422	`validation_error`	Ошибка валидации полей (например, вилка «до» меньше «от»)
429	`quota_exceeded`	Квота приглашений исчерпана
503	`registry_unavailable` , `idp_unavailable`	Недоступен внешний сервис

Пользователь

Профиль пользователя, выбор роли, согласия на обработку ПДн (152-ФЗ).

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/auth/config`	Параметры авторизации для фронтенда	200
`GET`	`/api/me`	Текущий пользователь и шаг онбординга	200, 401, 422
`GET`	`/api/me/consents`	История согласий	200, 401, 422
`POST`	`/api/me/consents`	Дать или отозвать согласие	200, 401, 422
`POST`	`/api/me/role`	Выбор роли при первом входе	200, 400, 401, 409, 422

Справочники

Специализации, грейды, навыки теста, отрасли, ФСП.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/catalog`	Справочники: специализации, грейды, навыки, отрасли, ФСП	200
`GET`	`/api/catalog/demo-vacancies`	Демонстрационные описания вакансий (для показа генерации тестов)	200
`GET`	`/api/catalog/validation`	Сводка результатов валидации механик (генерируется make validate)	200
`GET`	`/api/health`	Проверка работоспособности	200

Кандидат: профиль

Профиль, обязательный опрос, разбор PDF-резюме, PDF-профиль, приватность.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/candidate/dashboard`	Сводка кабинета кандидата	200, 401, 403, 422
`PUT`	`/api/candidate/privacy`	Настройки приватности	200, 401, 403, 422
`GET`	`/api/candidate/profile`	Мой профиль	200, 401, 403, 422
`PUT`	`/api/candidate/profile`	Заполнить профиль вручную	200, 401, 403, 422
`GET`	`/api/candidate/profile.pdf`	Стандартизированный PDF-профиль	200, 401, 403, 422
`POST`	`/api/candidate/resume`	Загрузить PDF-резюме и распознать поля	200, 400, 401, 403, 422
`PUT`	`/api/candidate/survey`	Пройти/обновить обязательный опрос	200, 401, 403, 422

Тестирование

Адаптивный тест на заявленный грейд (IRT/CAT, параметризованные семейства заданий).

Метод	Путь	Назначение	Коды ответов
GET	/api/assessment/attempts	История попыток	200, 401, 403, 422
POST	/api/assessment/attempts	Начать тест на грейд	201, 401, 403, 409, 422
GET	/api/assessment/attempts/{attempt_id}	Состояние попытки и текущее задание	200, 401, 403, 404, 422
POST	/api/assessment/attempts/{attempt_id}/abandon	Прервать попытку (засчитывается как неудачная)	200, 401, 403, 404, 409, 422
POST	/api/assessment/attempts/{attempt_id}/answer	Ответить на текущее задание	200, 401, 403, 404, 409, 422
GET	/api/assessment/eligibility	Доступные попытки и правила смены грейда	200, 401, 403, 422
POST	/api/assessment/preview	Как система формирует тест по описанию вакансии	200, 400, 404, 422

Работодатель

Компания, описание потребности, подборки с объяснениями, банк кандидатов.

Метод	Путь	Назначение	Коды ответов
GET	/api/employer/candidates	Поиск по банку кандидатов (порядок — сила подтверждённого профиля)	200, 401, 403, 409, 422
GET	/api/employer/candidates/{candidate_id}	Карточка кандидата (контакты — после принятия приглашения или отклика)	200, 401, 403, 404, 409, 422
GET	/api/employer/candidates/{candidate_id}/profile.pdf	PDF-профиль кандидата	200, 401, 403, 404, 409, 422
GET	/api/employer/candidates/{candidate_id}/test-photo	Кадр с теста с проктором — после принятого приглашения или отклика, при согласии кандидата	200, 401, 403, 404, 409, 422
GET	/api/employer/categories	Категоризированная база: специализация × грейд	200, 401, 403, 422
GET	/api/employer/company	Профиль компании	200, 401, 403, 422
PUT	/api/employer/company	Создать или обновить профиль компании	200, 401, 403, 422
POST	/api/employer/needs/parse	Разобрать описание потребности (NLP): навыки, специализация, грейд	200, 401, 403, 422
GET	/api/employer/searches	Мои подборки	200, 401, 403, 409, 422
POST	/api/employer/searches	Описать потребность и получить подборку	201, 400, 401, 403, 409, 422
DELETE	/api/employer/searches/{search_id}	Удалить подборку	204, 401, 403, 404, 409, 422
GET	/api/employer/searches/{search_id}	Подборка с ранжированием и объяснениями	200, 401, 403, 404, 409, 422
PUT	/api/employer/searches/{search_id}/filters	Уточнить фильтры (новая версия, прежние сохраняются)	200, 401, 403, 404, 409, 422
PUT	/api/employer/searches/{search_id}/fsp	ФСП-настройки подборки: засчитывать ФСП как опыт, уведомлять о новых участниках ФСП	200, 401, 403, 404, 409, 422
POST	/api/employer/searches/{search_id}/revert/{version}	Откатить фильтры к версии	200, 401, 403, 404, 409, 422
DELETE	/api/employer/searches/{search_id}/shortlist/{candidate_id}	Убрать из шорт-листа	204, 401, 403, 404, 409, 422
PUT	/api/employer/searches/{search_id}/shortlist/{candidate_id}	Закрепить кандидата в шорт-листе	204, 401, 403, 404, 409, 422

Приглашения

Выход на контакт по инициативе работодателя; статусы; обратная связь рынка.

Метод	Путь	Назначение	Коды ответов
GET	/api/candidate/invitations	Входящие приглашения	200, 401, 403, 422
GET	/api/candidate/invitations/{inv_id}	Открыть приглашение (статус → просмотрено)	200, 401, 403, 404, 422
POST	/api/candidate/invitations/{inv_id}/accept	Принять приглашение — контакты станут видны работодателю	200, 401, 403, 404, 409, 422
POST	/api/candidate/invitations/{inv_id}/decline	Отклонить приглашение с причиной	200, 401, 403, 404, 409, 422
GET	/api/employer/insights	Обратная связь рынка: причины отказов, ожидания по категориям	200, 401, 403, 409, 422
GET	/api/employer/invitations	Отправленные приглашения и их статусы	200, 401, 403, 409, 422
POST	/api/employer/invitations	Пригласить кандидата (вилка обязательна)	201, 401, 403, 404, 409, 422, 429
POST	/api/employer/invitations/{inv_id}/hired	Отметить, что кандидат вышел на работу (для воронки трудоустройства)	200, 401, 403, 404, 409, 422

Метод	Путь	Назначение	Коды ответов
`POST`	`/api/employer/invitations/{inv_id}/withdraw`	Отозвать приглашение	200, 401, 403, 404, 409, 422
`GET`	`/api/employer/salary-check`	Индикатор вилки до отправки приглашения	200, 401, 403, 404, 409, 422

Вакансии и отклики

Классический сценарий: публикация вакансий и самостоятельный отклик.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/candidate/applications`	Мои отклики и их статусы	200, 401, 403, 422
`POST`	`/api/employer/applications/{app_id}/status`	Сменить статус отклика	200, 401, 403, 404, 409, 422
`GET`	`/api/employer/vacancies`	Мои вакансии	200, 401, 403, 409, 422
`POST`	`/api/employer/vacancies`	Опубликовать вакансию (вилка обязательна)	201, 400, 401, 403, 409, 422
`PUT`	`/api/employer/vacancies/{vacancy_id}`	Редактировать вакансию	200, 400, 401, 403, 404, 409, 422
`GET`	`/api/employer/vacancies/{vacancy_id}/applications`	Отклики на вакансию	200, 401, 403, 404, 409, 422
`GET`	`/api/vacancies`	Лента вакансий (сортировка по соответствию профилю)	200, 401, 403, 422
`GET`	`/api/vacancies/{vacancy_id}`	Вакансия	200, 401, 403, 404, 422
`POST`	`/api/vacancies/{vacancy_id}/apply`	Откликнуться (контакты станут видны работодателю)	201, 401, 403, 404, 409, 422

Регулярные задания

Короткие задания от работодателей для актуальности профиля.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/candidate/tasks`	Мои регулярные задания (новое назначается раз в 2 недели)	200, 401, 403, 422
`POST`	`/api/candidate/tasks/{assignment_id}/submit`	Отправить решение	200, 400, 401, 403, 404, 409, 422
`POST`	`/api/employer/submissions/{assignment_id}/review`	Оценить решение по рубрике	200, 401, 403, 404, 409, 422
`GET`	`/api/employer/tasks`	Мои задания	200, 401, 403, 409, 422
`POST`	`/api/employer/tasks`	Создать регулярное задание	201, 401, 403, 409, 422
`GET`	`/api/employer/tasks/{task_id}/submissions`	Решения кандидатов	200, 401, 403, 404, 409, 422

ФСП

Привязка ФСП ID (OIDC) и синхронизация достижений из реестра ФСП.

Метод	Путь	Назначение	Коды ответов
`PUT`	`/api/fsp/achievements/{achievement_id}/project`	Ссылка на проект команды для достижения ФСП (видна работодателю в карточке)	200, 401, 403, 404, 422
`GET`	`/api/fsp/funnel`	Воронка трудоустройства участников ФСП (только агрегаты, для федерации)	200, 401, 403, 422
`GET`	`/api/fsp/growth`	План роста: что подтянуть по итогам теста и ближайшие соревнования ФСП	200, 401, 403, 422
`DELETE`	`/api/fsp/link`	Отвязать ФСП ID	200, 401, 403, 422
`GET`	`/api/fsp/link/callback`	Callback ФСП ID (браузерный редирект)	302, 422
`GET`	`/api/fsp/link/start`	Начать привязку ФСП ID (редирект на страницу входа ФСП ID)	200, 401, 403, 422
`POST`	`/api/fsp/sync`	Обновить достижения из реестра ФСП	200, 401, 403, 409, 422, 503

Примеры

Начать тест и ответить на задание:

```
POST /api/assessment/attempts
{"grade": "middle"}

→ 201 {"id": "…", "status": "in_progress", "item": {"position": 0, "kind": "text", "prompt": "Что выведет программа?…", "code":
"print(-30 // 7, …)", "time_limit": 150}}

POST /api/assessment/attempts/{id}/answer
{"position": 0, "value": "-5 5 -4", "time_ms": 41000, "tab_switches": 0}
```

Описать потребность и получить подборку:

```
POST /api/employer/searches
{"title": "Python-разработчик", "text": "Нужен бэкенд: Python, FastAPI, PostgreSQL. Опыт от 2 лет. Вилка 250-320 тыс ₽"}

→ 201 {"search": {"category": "Backend · Middle", …}, "categories": […], "results": [{"card": {"alias": "Backend #6306", "contacts":
null, …},
        "score": 0.70, "components": {"need_fit": 0.23, …}, "reasons": [{"kind": "positive", "text": "Подтверждено тестом: Python,
PostgreSQL"}, …]}]}
```

Пригласить кандидата (вилка обязательна):

```
POST /api/employer/invitations
{"candidate_id": "…", "title": "Python-разработчик", "description": "…", "salary_from": 250000, "salary_to": 320000,
 "contact_method": "telegram", "contact_value": "@hr_company"}
```

Используемые библиотеки и компоненты

> Сгенерировано `docs/tools/gen_docs.py` из `uv.lock` и `frontend/package-lock.json`. Все компоненты — открытое ПО (MIT / Apache-2.0 / BSD / ISC / OFL / LGPL с динамической линковкой).

Инфраструктура

Компонент	Версия	Лицензия	Назначение
Python	3.14 (`python:3.14-slim`; код совместим с 3.12+)	PSF	Образ бэкенда и реестра ФСП
uv	0.12 (`ghcr.io/astral-sh/uv:0.12`)	Apache-2.0 / MIT	Установка зависимостей по `uv.lock`
PostgreSQL	18 (`postgres:18-alpine`)	PostgreSQL License	Основная БД платформы и БД Keycloak
Keycloak	26.8 (`quay.io/keycloak/keycloak:26.8`)	Apache-2.0	Аутентификация, подтверждение email; имитация ФСП ID
Mailpit	1.31 (`axllent/mailpit:v1.31`)	MIT	Перехват писем в демо (подтверждение email, уведомления)
nginx	1.30 (`nginx:1.30-alpine`)	BSD-2	Раздача фронтенда и моделей проктора, прокси `/api`
Node.js	24 (`node:24-alpine`, только сборка)	MIT	Сборка фронтенда
Docker Compose	v2	Apache-2.0	Оркестрация стека
JetBrains Mono	2.304	OFL-1.1	Фирменный шрифт ФСП (PDF, интерфейс)
Модели Human: BlazeFace, FaceMesh, FaceRes, Liveness, Antispoof, CenterNet	из `@vladmandic/human` 3.3	MIT	Цифровой проктор: лицо, взгляд, отпечаток лица, живость, телефон; в браузере кандидата
EfficientDet-Lite0 (COCO, float16)	из MediaPipe Tasks Vision 1.1	Apache-2.0	Цифровой проктор: второй детектор телефона, в браузере кандидата
rubert-tiny2 (cointegrated)	—	MIT	Локальные эмбеддинги — только при сборке с `WITH_EMBEDDINGS=1` (по умолчанию TF-IDF)

Python (бэкенд и ML-ядро — образ `backend`)

Пакет	Версия	Лицензия	Назначение
fastapi	0.143.0	MIT	веб-фреймворк API, OpenAPI
uvicorn	0.54.0	BSD-3	ASGI-сервер
sqlalchemy	2.1.4	MIT	ORM
alembic	1.20.0	MIT	миграции БД
psycopg	3.3.6	LGPL-3 (динамическая линковка)	драйвер PostgreSQL
pydantic	2.14.0	MIT	валидация и схемы
pydantic-settings	2.15.0	MIT	конфигурация из окружения
pyjwt	2.15.1	MIT	проверка JWT (OIDC)
httpx	0.28.1	BSD-3	HTTP-клиент (реестр ФСП, Keycloak, webhook)
python-multipart	0.0.32	Apache-2.0	загрузка файлов
fpdf2	2.8.9	LGPL-3	PDF-профиль, синтетические резюме
email-validator	2.3.0	Unlicense	валидация email
numpy	2.5.3	BSD-3	IRT/CAT, ранжирование
scipy	1.18.1	BSD-3	оптимизация в MML-EM
pyyaml	6.0.3	MIT	справочники
rapidfuzz	3.14.6	MIT	нечёткий поиск технологий
scikit-learn	1.9.1	BSD-3	TF-IDF эмбеддинги
lightgbm	4.7.0	MIT	LTR-модель подбора (эксперимент бенчмарка, в продукте не используется)
cryptography	50.0.2	Apache-2.0 / BSD	AES-256-GCM для эталонов лица (проктор)
sentence-transformers	6.1.0	Apache-2.0	эмбеддинги rubert-tiny2 (только при `WITH_EMBEDDINGS=1`)
pdfplumber	0.11.10	MIT	извлечение текста из PDF
pandas	3.0.6	BSD-3	задания по pandas (исполнение сниппетов)

`sentence-transformers` ставится в образ только при `WITH_EMBEDDINGS=1`; по умолчанию его нет.

Python — разработка и валидация (в Docker-образ не входят)

Пакет	Версия	Лицензия	Назначение
matplotlib	3.11.2	PSF-based	графики валидации
pytest	9.1.1	MIT	тесты
ruff	0.16.10	MIT	линтер/форматтер
markdown	3.11	BSD-3	сборка PDF-документации

Фронтенд (npm, собирается в образ `frontend`)

Пакет	Версия	Лицензия	Назначение
@fontsource-variable/inter	5.3.0	OFL-1.1	шрифт текста
@fontsource/jetbrains-mono	5.3.0	OFL-1.1	фирменный шрифт ФСП
@mediapipe/tasks-vision	1.1.0	Apache-2.0	цифровой проктор: детектор телефона EfficientDet-Lite0
@tanstack/react-query	5.104.1	MIT	кеш запросов к API
@vladmandic/human	3.3.6	MIT	цифровой проктор в браузере: лицо, взгляд, телефон, эмбеддинг, liveness
clsx	2.1.1	MIT	классы
keycloak-js	26.2.4	Apache-2.0	OIDC-клиент (PKCE)
lucide-react	1.54.0	ISC	иконки
react	19.3.0	MIT	UI
react-dom	19.3.0	MIT	UI
react-router-dom	7.18.4	MIT	маршрутизация
@tailwindcss/vite	4.3.3	MIT	сборка стилей
@types/node	24.19.1	MIT	типы
@types/react	19.3.0	MIT	типы
@types/react-dom	19.3.0	MIT	типы
@vitejs/plugin-react	6.1.2	MIT	сборка React
oxlint	1.87.0	MIT	линтер
tailwindcss	4.3.3	MIT	стили
typescript	7.0.2	Apache-2.0	типизация
vite	8.3.4	MIT	сборка фронтенда

Все модели работают локально: серверные — на CPU внутри контейнера `backend`, модели проктора — в браузере кандидата (раздаются своим nginx, без внешних CDN). Внешние сервисы и ключи не нужны, персональные данные не покидают контур развёртывания.

Сборка, развёртывание и локальный запуск

1. Быстрый старт (Docker Compose, ~5 минут)

Требования: 2 vCPU, 4 ГБ RAM, Docker 24+ с Compose v2; Linux, macOS или Windows (Docker Desktop / WSL2). Свободные порты 18000, 18025, 18080, 18088, 18090. Интернет нужен только для скачивания образов и пакетов при первой сборке; ключи облачных сервисов не нужны.

```
git clone https://github.com/Izymka/fsp-job-board.git && cd fsp-job-board
cp .env.example .env          # для локального запуска можно ничего не менять
make up                       # docker compose up -d --build + HTTP для админки Keycloak; первая сборка ~3-5 мин
docker compose ps             # дождитесь healthy у postgres и keycloak (~1-2 мин)
```

Без `make` (например, в Windows без WSL): `docker compose up -d --build` — платформа работает так же, не будет только HTTP-доступа к админке Keycloak (см. «Диагностика»).

Адрес	Что там
http://localhost:18088	Платформа
http://localhost:18088/api/docs	Swagger UI (OpenAPI), ReDoc — `/api/redoc`
http://localhost:18080	Keycloak (админ: `admin` / `admin` из `.env`)
http://localhost:18025	Mailpit — письма подтверждения email и уведомления
http://localhost:18090/docs	Реестр ФСП (имитация)
http://localhost:18000/api/docs	API напрямую, минуя nginx

Порт платформы меняется переменной `HTTP_PORT` в `.env`.

`make up` после запуска контейнеров разрешает HTTP для админки Keycloak на демо-стенде (realm `master`; realm-ы платформы уже настроены на `sslRequired: none` — в продакшене включите TLS на прокси и `sslRequired: external`).

При старте бэкенд применяет миграции (`alembic upgrade head`) и загружает демо-данные (`python -m app.seed`): ~300 кандидатов, прошедших симулированный тест, 7 компаний с вакансиями, регулярные задания, историю приглашений.

Демо-аккаунты (тестовые учётные данные стенда)

Логин	Пароль	Роль
`employer@demo.fsp`	`demo12345`	Работодатель, компания «Рубль+»
`maria@demo.fsp`	`demo12345`	Кандидат Backend · Middle с привязанным ФСП ID
`candidate@demo.fsp`	`demo12345`	Новый кандидат — для показа онбординга
`federation@demo.fsp`	`demo12345`	Федерация ФСП: воронка трудоустройства участников (только агрегаты)
ФСП ID: `fsp-000123`	`demo12345`	Участник ФСП с достижениями (для «Привязать ФСП ID»)
ФСП ID: `fsp-000999`	`demo12345`	Участник без истории ФСП

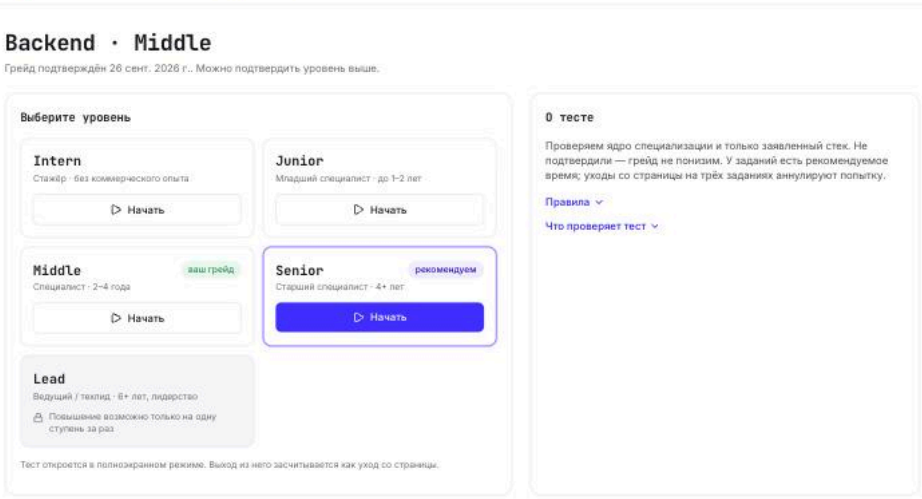
Новая регистрация: «Регистрация» → форма Keycloak → письмо в Mailpit (http://localhost:18025) → ссылка подтверждения → выбор роли.

2. Сценарий проверки (ТЗ §3.3)

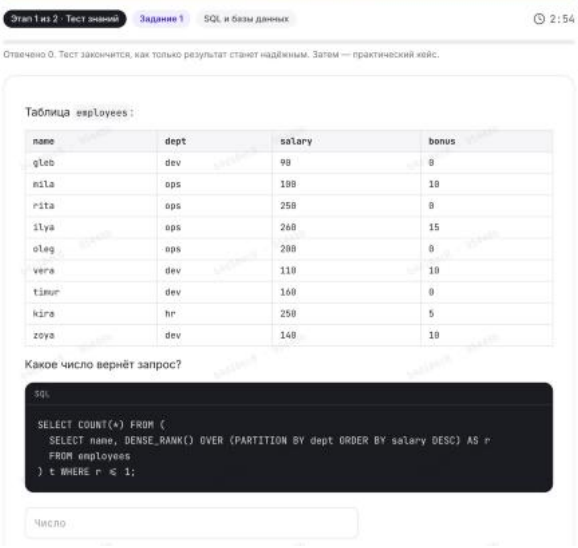
1. **Кандидат:** войти `candidate@demo.fsp` → роль «Ищу работу» + согласие → опрос (Backend, стек, Middle, ожидания) → «Тест и грейд» → пройти тест → категория «Backend · Middle» (или тест на уровень ниже).
2. **Работодатель:** войти `employer@demo.fsp` → «Подборки» → «Новая подборка» → вставить текст потребности (кнопка «Вставить пример») → «Показать подборку» → рекомендованные категории, кандидаты с объяснением → фильтр «есть ФСП» → «Сбросить уточнения» → «Пригласить» (индикатор вилки).
3. **Контакт:** войти `maria@demo.fsp` или кандидатом из шага 1 → «Приглашения» → открыть (статус «просмотрено») → «Принять» → у работодателя в «Приглашения» появились ФИО и контакты.
4. **Отклик:** кандидат → «Вакансии» → откликнуться → работодатель → «Вакансии» → «Отклики» (контакты открыты сразу).
5. **Федерация:** войти `federation@demo.fsp` → воронка трудоустройства участников ФСП в сравнении с кандидатами без ФСП.
6. **Механики:** публичные страницы «Тест из описания вакансии» (`/test-demo`) и «Механика подбора и тестирования» (`/how-it-works`).

Как это выглядит

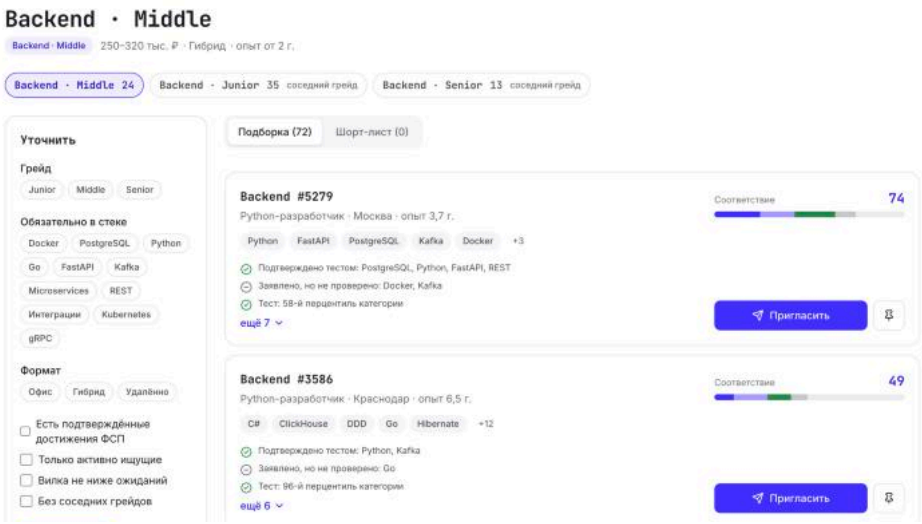
Кандидат — выбор уровня теста. Подтверждённый грейд, рекомендованный уровень для повышения; правила и состав теста свёрнуты под «Подробнее».



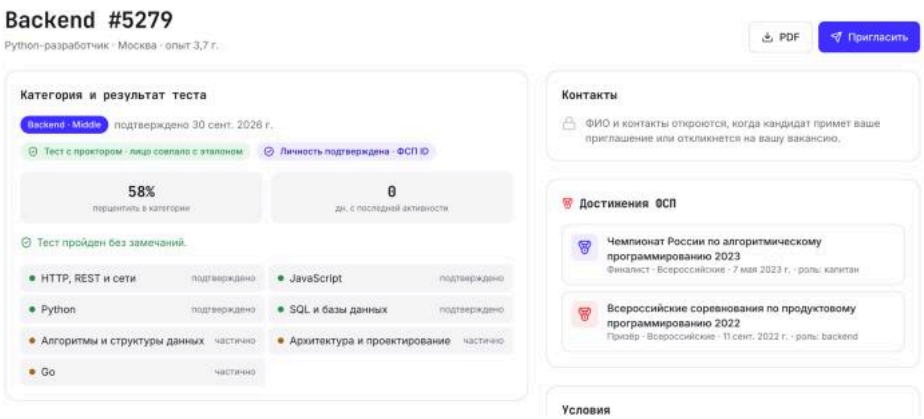
Задание теста. Генерируемый экземпляр (у каждого кандидата свои данные), водяной знак с кодом попытки, рекомендуемое время.



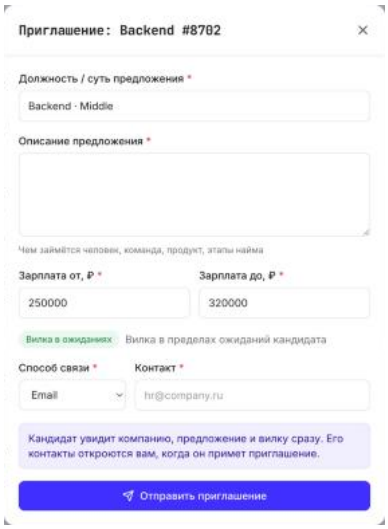
Работодатель — подборка. Рекомендованные категории (целевая и соседние грейды), фильтры без потери исходной выдачи, у каждого кандидата — балл соответствия и объяснение: что подтверждено тестом, что лишь заявлено.



Карточка кандидата. Обезличенный профиль: категория, «Тест с проктором · лицо совпало с эталоном», навыки по результатам теста, достижения ФСП; контакты скрыты до принятия приглашения.



Приглашение. Вилка «от–до» обязательна; ещё до отправки видно, укладывается ли она в ожидания кандидата (без раскрытия чисел).



3. Развёртывание на сервере

VPS: 2–4 vCPU, 4–8 ГБ RAM, Ubuntu 22.04+, Docker. Домены, например `jobs.example.ru` (платформа) и `auth.example.ru` (Keycloak).

1. В `.env`: `PUBLIC_URL=https://jobs.example.ru`, `KEYCLOAK_PUBLIC_URL=https://auth.example.ru`, сменить `POSTGRES_PASSWORD`, `KEYCLOAK_ADMIN_PASSWORD`, `ASSESSMENT_SECRET`, `WEBHOOK_SECRET`, `FSP_ID_CLIENT_SECRET`, `FSP_REGISTRY_API_KEY`, `BIOMETRIC_KEY` и задать `ENVIRONMENT=production`: с ним бэкенд не стартует, если остались секреты из `.env.example` или `AUTH_MODE=dev`. Камера и микрофон для проктора доступны браузеру только по HTTPS или на localhost.
2. В `keycloak/realm-*.json` добавить redirect URI для домена (`https://jobs.example.ru/*`, `https://jobs.example.ru/api/fsp/link/callback`) и настроить SMTP реального почтового сервера; удалить демо-пользователей.
3. TLS — обратный прокси (Caddy/nginx) перед `frontend` (порт 18088) и `keycloak` (порт 18080). Пример Caddyfile: `jobs.example.ru { reverse_proxy localhost:18088 } auth.example.ru { reverse_proxy localhost:18080 }`
4. `docker compose up -d --build`. Для Keycloak в продакшене используйте `start` вместо `start-dev` и `KC_PROXY_HEADERS=xforwarded`.
5. Резервное копирование: `docker compose exec postgres pg_dump -U fsp fsp_jobs > backup.sql`. Данные PostgreSQL 18 лежат в томе `pgdata18`.

Сборочный флаг `WITH_EMBEDDINGS=1` добавляет в образ бэкенда локальную модель эмбеддингов rubert-tiny2 (CPU, +~700 МБ к образу, ~150 МБ RAM); по умолчанию выключен — семантическая близость считается TF-IDF.

Диагностика

- `docker compose logs -f backend keycloak` — логи; у бэкенда в начале должно быть «Готово: 300 кандидатов…».
- Страница Keycloak «Требуется HTTPS» — выполните `make kc-demo` (или задайте `sslRequired` в realm).
- `docker pull` / сборка «висит» на загрузке метаданных образа в Docker Desktop — зависает помощник учётных данных (`credsStore: desktop` в `~/.docker/config.json`): перезапустите Docker Desktop или соберите с чистым конфигом `DOCKER_CONFIG=$(mktemp -d) docker compose build`.

4. Локальная разработка без Docker

Требования: Python 3.12+ (в Docker — 3.14), uv, Node.js 22.12+ (в Docker — 24).

```
uv sync --all-packages          # Python-зависимости всех модулей
make dev                        # всё сразу: реестр, API и фронтенд в одном терминале (Ctrl+C — остановить)
```

```
# или по отдельности, каждый в своём терминале:
make dev-registry               # реестр ФСП на :8090 (отдельный терминал)
make dev-backend                # API на :8000: SQLite, AUTH_MODE=dev (вход без Keycloak), seed
make dev-frontend               # фронтенд на :5173 (прокси /api → :8000)
```

В режиме `AUTH_MODE=dev` страница входа предлагает демо-аккаунты без пароля — **только для локальной отладки**; привязка ФСП ID в этом режиме — демо-кнопкой по номеру.

5. Тесты, линтер, валидация

```
make test        # 1030 тестов ML-ядра + 48 тестов API + сборка фронтенда
make lint        # ruff
make validate    # процедура валидации → docs/validation.md, docs/img/*.png (~6 мин)
make docs       # OpenAPI, api.md, components.md → документация одним PDF (docs/fsp-job-board-docs.pdf)
```

Концепции развития: защита от недобросовестных работодателей и интеграция с ATS

ТЗ относит эти механизмы к развитию продукта («будет плюсом»): ожидается концепция и архитектурный подход. Часть основы уже реализована в MVP — в таблице отмечено «реализовано».

1. Защита от фиктивных вакансий и недобросовестных работодателей

Принципы

- Кандидат должен видеть, кто ему пишет, *до* раскрытия своих данных.
- Каждое действие работодателя оставляет измеримый след, из которого строится репутация.
- Санкции градуированы: от предупреждения до блокировки, с апелляцией.

Слои защиты

Слой	Механизм	Статус
Прозрачность условий	Вилка «от–до» обязательна в вакансии и приглашении (валидация API + CHECK в БД)	реализовано
Защита ПДн кандидата	Контакты открываются только после принятия приглашения / отклика; журнал раскрытия	реализовано
Ограничение рассылок	Квота приглашений в неделю; запрет повторного приглашения при активном	реализовано
Репутация	Кандидат видит у компании: приглашений за 90 дней, долю принятых, долю отвеченных откликов	реализовано
Сигнал от кандидатов	Причины отказов (в т.ч. «компания/отрасль») агрегируются	реализовано
Верификация юрлица	Проверка ИНН/ОГРН по ЕГРЮЛ (API ФНС), статус «действующее», совпадение названия; поле `inn` уже есть	концепция
Подтверждение домена	Письмо на корпоративный адрес домена сайта компании; бейдж «домен подтверждён»	концепция
Модерация по риску	Автоматические флаги: массовые приглашения с вилкой сильно ниже рынка категории (по данным «обратной связи рынка»), всплеск отказов «компания», жалобы, свежая регистрация + высокая активность → ручная проверка, временная заморозка квоты	концепция
Жалобы	Кнопка «пожаловаться» в приглашении с типами (фейковая вакансия, просьба оплатить обучение, несоответствие условий)	концепция
Анти-скам паттерны	Детекция в тексте приглашения: просьбы предоплаты, «обучение за свой счёт», внешние ссылки на сбор данных	концепция
Связь с ФСП	Партнёры ФСП получают бейдж «партнёр Федерации» по реестру партнёров	концепция

Модель репутации

`trust = f(верификация юрлица, подтверждение домена, acceptance rate, response rate к откликам, доля отказов «компания», жалобы, возраст аккаунта)` — рассчитывается ежедневно; при низком доверии: сниженная квота, пометка «новая компания», обязательная проверка вакансий модератором.

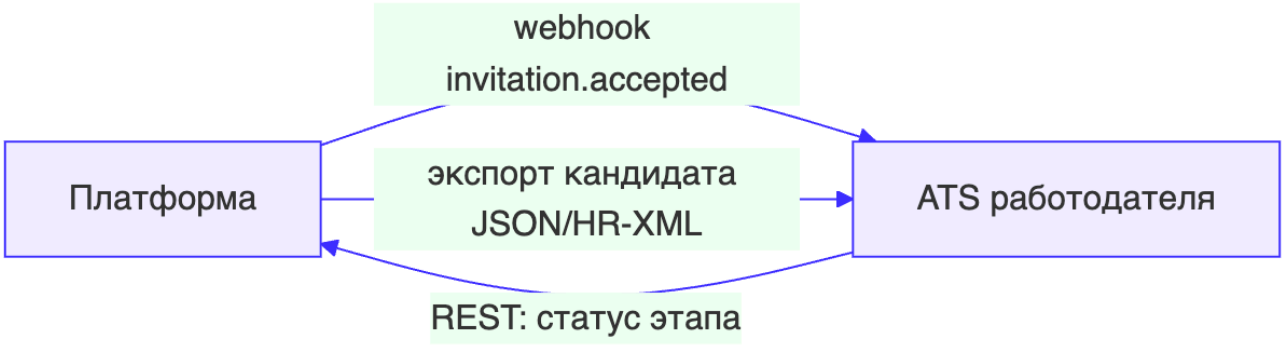
2. Интеграция с ATS работодателя

Что уже есть

- **Webhook** `invitation.accepted` на URL из профиля компании — отправляется только после согласия кандидата (принятия приглашения). Тело: `invitation_id`, `accepted_at`, `title`, `salary`, `vacancy_id` и `candidate` — `id`, `name`, `email`, `phone`, `telegram`, `github`, `category`, `specialization`, `grade`, `fsp_id` (если кандидат не скрыл ФСП). Имя и почта берутся из профиля, а если кандидат их не заполнил — из учётной записи, поэтому работодатель всегда получает способ связаться. Тело подписано: `X-FSP-Signature: sha256=<HMAC-SHA256(ключ, тело)>`; ключ компании (`webhook_signing_key`) виден работодателю в профиле компании и выводится из секрета платформы `WEBHOOK_SECRET`. Отправка — только на публичные http(s)-адреса, без редиректов: адрес задаёт работодатель, и запросы во внутреннюю сеть стека отклоняются.
- **REST API** с OpenAPI-описанием: подборки, кандидаты, приглашения, отклики — интегратор может работать от имени сервисного пользователя работодателя.
- **PDF-профиль кандидата** в стандартизированном виде.

Развитие

1. Ретраи webhook с экспоненциальной задержкой, журнал доставок, события `application.created`, `invitation.declined`, `candidate.updated`.
2. **Сервисные клиенты** (OAuth2 client credentials в Keycloak) со scope `jobs:read`, `invitations:write`.
3. **Экспорт кандидата** в форматах JSON Resume / HR-Open (HR-XML) — с обезличиванием до согласия.
4. **Коннекторы** для популярных российских ATS (Huntflow, Talantix, E-Staff, Поток): маппинг кандидата и статусов, двусторонняя синхронизация этапов (статус в ATS «интервью» → статус отклика «интервью» на платформе, кандидат видит прогресс).
5. **Импорт вакансий** из ATS с обязательной проверкой наличия вилки.



3. Планы развития после хакатона

Направление	Что сделать	Опора в MVP
Задачи с кодом	Кандидат сам пишет код, решение исполняется в изолированной песочнице и проверяется автотестами; результат входит в тот же IRT/CAT как отдельный формат задания	Генераторы уже вычисляют ответы исполнением кода; в CI задания сверяются с настоящими компиляторами
Настоящий ФСП ID	Подключить ФСП ID (Keycloak) и реестр достижений по предложенной схеме данных	Привязка — стандартный OIDC code flow; переход — смена адресов, клиента и маппинга claim `fsp_id` (раздел «Интеграция с ФСП»)
Пилот	Реальные кандидаты и эксперты: уточнить сложность заданий (MML-EM) и проверить качество подбора на принятых приглашениях	Протокол пилота (раздел «Живой пилот»), онлайн-перекалибровка и мониторинг утечек уже работают
Защита от фейков	Проверка компаний по ИНН/ОГРН, чтобы отсечь фиктивные вакансии	Поле `inn` в профиле компании, квоты и репутация (§1)
Второй ракурс проктора	Камера телефона кандидата как второй ракурс: фронтальная камера ноутбука имеет слепые зоны, в которых можно сфотографировать задание	Проктор, водяной знак с кодом попытки на каждом задании

Протокол живого пилота тестирования

Синтетическая валидация (validation.md) проверяет механику при известных допущениях. Живой пилот проверяет сами допущения: реальную сложность семейств, время ответа, согласие грейда с экспертной оценкой.

Участники

15–20 человек: команда, знакомые разработчики и аналитики разных грейдов (минимум по 3 на Junior / Middle / Senior), желательно 2–3 участника соревнований ФСП.

Процедура

1. **Экспертная оценка до теста.** Два эксперта независимо оценивают грейд участника по короткому структурированному интервью (15 минут, единый опросник по специализации) — эталон для сравнения. Эксперты не видят результатов теста.
2. **Тест.** Участник проходит опрос и тест на самооценённый грейд на стенде (обычный пользовательский путь).
3. **Ретест.** Через 1–3 дня — повторный тест (другие варианты заданий) на тот же грейд; регистрируется решение и θ.
4. **Анкета UX** (5 вопросов, шкала 1–5): понятность заданий, достаточность времени, ощущение честности, понятность результата, желание пройти повышение.

Метрики

- Согласие грейда теста с экспертным: доля совпадений, ±1 ступень, Cohen's κ; согласие двух экспертов между собой (эталон «человеческой» надёжности).
- Тест–ретест: совпадение решения, корреляция θ.
- По семействам: доля верных ответов, точечно-бисериальная корреляция, медианное время — семейства с аномалиями идут на доработку; ответы пилота — первые данные для перекалибровки (MML-EM).
- Технические: число заданий до решения, длительность теста, ошибки интерфейса.

Обработка данных

Участники дают согласие на обработку ПДн; в отчёт идут только обезличенные агрегаты. Результаты — раздел «Результаты» ниже.

Результаты

Статус на 21.10.2026: пилот тестирования с живыми участниками не проведён — выше протокол, по которому он будет проведён с ФСП. Числа для слайдов берутся только из разделов с измерениями: validation.md (синтетическая популяция со скрытой истиной), bench2-results.md (слепая разметка живых текстов резюме), bench3-results.md (реальные вакансии).

Что измеряем	Где	Результат
Согласие грейда теста с экспертами (живые люди)	пилот, протокол выше	не проведено
Тест–ретест на живых людях	пилот, протокол выше	не проведено
Время ответа и сложность семейств на живых людях	пилот, протокол выше	не проведено
Точность грейда, завышение, ретест (синтетика)	validation.md	72.3% точно (5 сидов, 71.5–73.2%), завышение 4.2%, взв. к ретеста 0.83, 29 заданий за онбординг
Качество подборки по слепой разметке (живые тексты резюме)	bench2-results.md	nDCG@10: v5 0.789, v6 0.812, **v7 0.817**
Разбор реальных вакансий нашим NLP (Хабр Карьера)	bench3-results.md	специализация 9/10; грейд на 50 вакансиях: dev 25/31, отложенный test 14/19, при высокой уверенности 12/13
Выдача на реальных вакансиях: участники ФСП без стажа	bench3-results.md	в v7 поднялись относительно v5 в 9 из 18 появлений в выдаче